CABS: Einleitung

Computer-Architekturen und Betriebssysteme
Teilbereich: Digitaltechnik

9. Februar 2026

Diese Folien basieren auf dem Skriptum [1]

Organisatorisches

- Die Gesamtnote CABS ergibt sich durch
  - KIF: 2 Stunden (hier) und 7 weiteren Stunden CABS
  - AIF/BIF: 1 Stunde (hier) und zwei weiter Stunden
  - **Diese Doppelstunde:**
    1. "Aussagenlogik" (1. Sem.)
    2. **"Digitaltechnik" (2. Sem.)**
- KIF: Alle Teile müssen positiv bestanden werden um positive Gesamtnote zu erhalten.
- Im Unterricht besteht Anwesenheitspflicht
- Die Anwesenheit bestätigen Sie in jeder Einheit durch Unterschrift auf einer Anwesenheitsliste

Beurteilung

- Zwei schriftliche Leistungsüberprüfungen (SLÜs) pro Semester
- Bei jeder SLÜ sind 40 Punkte zu erreichen.
- Semesternote (Teilbereich):

$$\frac{\text{Punkte 1. SLUE} + \text{Punkte 2. SLUE}}{2}$$

mit folgendem Punkteschlüssel:

Punkte	Note
$[0, 20]$	Nicht Genügend
$(20, 25]$	Genügend
$(25, 30]$	Befriedigend
$(30, 35]$	Gut
$(35, 40]$	Sehr Gut

Stoffübersicht

- Elektrische Schaltkreise
- Elektronische Schaltkreise
  - Gatter
  - Schaltungen: Halbaddierer, Volladdierer
  - ...
- Sequentielle Schaltungen
  - RS-Flip-Flop
  - JK-Flip-Flop
  - Getaktete Flip-Flops
  - Schieberegister
  - ...
- Zählerschaltungen
  - Vorwärts-/Rückwärtszähler
- Zahlensysteme
  - Zweier-Komplement-Darstellung (Festkommaarithmetik)
  - Fließkommaarithmetik
- Prädikatenlogik

Weiterführende Inhalte (*)

- Manche Inhalte sind speziell für besonders Interessierte gedacht
- Diese Inhalte zählen nicht zum Kernstoffgebiet, und müssen somit nicht gelernt/gekonnt/verstanden werden
- Die Kennzeichnung dieser Inhalte erfolgt durch die blaue Titel- und Fußzeile in den Folien!

Literaturübersicht I

- **Berger, Krieger, Mahr: "Grundlagen der elektronischen Datenverarbeitung", Skriptum**
- Jürgen Reichardt: "Digitaltechnik", De Gruyter Oldenbourg, 4. Auflage, 2017
- Blieberger et al.: "Informatik", Springer, 3. Auflage, 1996
- Patterson, Hennessy: "Computer Organization & Design", Second Edition, Morgan Kaufmann, 1998
- Wikipedia (Englisch): `https://en.wikipedia.org/`
- Wikipedia (Deutsch): `https://de.wikipedia.org/`

Notation

Junktoren

- Negation: $\neg a$, oft auch geschrieben mit $\overline{A}$
- Konjunktion: $a \wedge b$, alternativ $A \cdot B$
- Disjunktion: $a \vee b$, alternativ $A + B$
- Implikation: $a \rightarrow b$
- Äquivalenz (XNOR): $a \leftrightarrow b$
- Antivalenz (XOR): $a \veebar b$, alternativ $a \nleftrightarrow b$ oder $a \oplus b$
- Peirce (NOR): $a \downarrow b$, alternativ $a \overline{\vee} b$
- Sheffer (NAND): $a \uparrow b$, alternativ $a \overline{\wedge} b$

Äquivalenz

- als Junktor: $\leftrightarrow$
- ist Tautologie: $\equiv$
- als Äquivalenzumformung: $\Leftrightarrow$

Rechenregeln

Anmerkung: Bei Rechenregeln die für Konjunktion und Disjunktion analog definiert sind, ist in diesem Wiederholungs-Abschnitt nur eine der beiden Regeln angeführt.

- De Morgan:
  - $\neg(a \wedge b) \equiv \neg a \vee \neg b$
  - alternative Schreibweise: $\overline{(A \wedge B)} \equiv \overline{A} \vee \overline{B}$
- Distributivgesetz:

$$a \wedge (b \vee c) \equiv (a \wedge b) \vee (a \wedge c)$$

- Implikation:
  - $a \rightarrow b \equiv \neg a \vee b$
  - $a \rightarrow b \equiv \neg b \rightarrow \neg a$ (Kontraposition)
  - $a \wedge (a \rightarrow b) \rightarrow b \equiv \mathrm{w}$ (Modus ponens)
  - $(a \rightarrow b) \wedge \neg b \rightarrow \neg a \equiv \mathrm{w}$ (Modus tollens)

Rechenregeln (Folgerungen)

- Idempotenz:
  - $a \vee a \equiv a$
  - $a \wedge a \equiv a$
- Absorptionsgesetz/Verschmelzungsgesetz:
  - $a \wedge (a \vee b) \equiv a$
  - $a \vee (a \wedge b) \equiv a$
- **Ad**sorptionsgesetz:
  - $a \wedge (\neg a \vee b) \equiv a \wedge b$
  - $a \vee (\neg a \wedge b) \equiv a \vee b$
- Nachbarschaftsgesetze:
  - $(a \wedge b) \vee (a \wedge \neg b) \equiv a$
  - $(a \vee b) \wedge (a \vee \neg b) \equiv a$

Dualitätsprinzip

Symmetrie zwischen $\wedge$ und $\vee$:

Definition

Dualitätsprinzip Jede *Boole'sche* Aussage besitzt eine dazu duale, gleichwertige Aussage, wenn man $\wedge$ und $\vee$, sowie w und f vertauscht.

a	b	$a \wedge b$
f	f	f
f	w	f
w	f	f
w	w	w

Dual
$\Leftrightarrow$

a	b	$a \vee b$
w	w	w
w	f	w
f	w	w
f	f	f

Weitere Eigenschaften

$\wedge$:	$a \wedge \mathrm{w} \equiv a$	$a \wedge \mathrm{f} \equiv \mathrm{f}$	$a \wedge a \equiv a$	$a \wedge \neg a \equiv \mathrm{f}$
$\vee$:	$a \vee \mathrm{w} \equiv \mathrm{w}$	$a \vee \mathrm{f} \equiv a$	$a \vee a \equiv a$	$a \vee \neg a \equiv \mathrm{w}$
$\underline{\vee}$:	$a \underline{\vee} \mathrm{w} \equiv \neg a$	$a \underline{\vee} \mathrm{f} \equiv a$	$a \underline{\vee} a \equiv \mathrm{f}$	$a \underline{\vee} \neg a \equiv \mathrm{w}$
$\leftrightarrow$:	$a \leftrightarrow \mathrm{w} \equiv a$	$a \leftrightarrow \mathrm{f} \equiv \neg a$	$a \leftrightarrow a \equiv \mathrm{w}$	$a \leftrightarrow \neg a \equiv \mathrm{f}$

NAND, NOR, XNOR

Definitionen:

- NAND: $a \uparrow b := \neg(a \wedge b)$
- NOR: $a \downarrow b := \neg(a \vee b)$
- XNOR: $a \leftrightarrow b$ (entspricht Äquivalenz)

NAND, NOR, XNOR

Beispiel: wichtige Zusammenhänge

- $\neg a \equiv a \downarrow a \equiv a \uparrow a$
- $a \vee b \equiv (a \downarrow b) \downarrow (a \downarrow b)$
- $a \wedge b \equiv (a \downarrow a) \downarrow (b \downarrow b)$
- $a \wedge b \equiv (a \uparrow b) \uparrow (a \uparrow b)$
- $a \vee b \equiv (a \uparrow a) \uparrow (b \uparrow b)$
- $a \rightarrow b \Leftrightarrow \neg a \vee b \Leftrightarrow \neg(a \wedge \neg b) \Leftrightarrow \neg(a \wedge (b \uparrow b)) \Leftrightarrow a \uparrow (b \uparrow b)$

SOP vs. POS

Aufgrund der Schreibweise $A \cdot (B + C)$ für $a \wedge (b \vee c)$ existieren auch die Bezeichnungen *Sum of Products* (SOP) bzw. *Product of Sums* (POS) für DNF bzw. KNF. [1]

	XOR: $A \nleftrightarrow B$	XNOR: $A \leftrightarrow B$
SOP:	$(\overline{B} \cdot A) + (B \cdot \overline{A})$	$(\overline{B} \cdot \overline{A}) + (B \cdot A)$
POS:	$(\overline{B} + \overline{A}) \cdot (B + A)$	$(\overline{B} + A) \cdot (B + \overline{A})$

[1] Die alternativen Schreibweisen wurden hier vorgestellt, um das Lesen von Dokumenten oder Verwenden von Software wo diese Notationen verwendet werden zu erleichtern. In weiterer Folge werden wir jedoch die ursprünglich eingeführte Notation weiter verwenden.

Übungsaufgaben 0.1.1 - 0.1.3

1. Zeigen Sie (durch Umformung) die Distributivgesetze für XOR und XNOR:
   - $(a \wedge b) \veebar (a \wedge c) \equiv a \wedge (b \veebar c)$
   - $(a \wedge b) \leftrightarrow (a \wedge c) \equiv a \wedge (b \leftrightarrow c)$
2. Zeigen Sie (durch Umformung) das Adsorptionsgesetzt für XOR und XNOR:[a]
   - $A \cdot (\overline{A} \nleftrightarrow B) \equiv A \cdot B$
   - $A + (\overline{A} \leftrightarrow B) \equiv A + B$
3. Zeigen Sie (durch Umformung) die De Morgan'schen Gesetze für XOR und XNOR:
   - $\neg(a \veebar b) \equiv \neg a \leftrightarrow \neg b \equiv a \leftrightarrow b$
   - $\neg(a \leftrightarrow b) \equiv \neg a \veebar \neg b \equiv a \veebar b$

[a] Hier nochmals zur Übung mit alternativer Schreibweise

Übungsaufgaben 0.2.1 - 0.2.2

1. Vereinfachen Sie den Ausdruck

$$(\neg\neg p \wedge \neg q) \vee (\neg q \wedge \neg r \wedge p) \vee (r \vee (r \wedge \neg q \wedge p))$$

2. Zeigen Sie, dass die folgende Formel eine Tautologie ist:

$$(r \wedge \neg s) \vee \neg(((p \wedge \neg q) \vee (\neg r \wedge p \wedge \neg q)) \wedge (\neg(p \wedge \neg q) \vee (r \wedge \neg s)))$$

Übungsaufgaben (NAND/NOR) 0.3.1 - 0.3.2

1. Stellen Sie $a \to b$ mittels $\downarrow$ dar
2. Stellen Sie mittels $\uparrow$ dar:

$$(q \to r) \to (p \wedge (q \to \neg s))$$

3. Stellen Sie $\neg(p \wedge q) \to (p \wedge \neg r)$ mittels $\downarrow$ dar.
4. Stellen Sie $(\neg p \wedge \neg q) \vee (p \wedge q) \vee r \vee s$ mittels $\uparrow$ (und gegebenenfalls $\neg$) dar.

Wichtige Äquivalenzen

$a \equiv a$	Reflexivität der Äquivalenz
$\neg\neg a \equiv a$	Gesetz der doppelten Negation
$a \vee a \equiv a$	Idempotenz der Disjunktion
$a \vee \mathrm{w} \equiv \mathrm{w}$	
$a \vee \mathrm{f} \equiv a$	
$a \vee b \equiv b \vee a$	Kommutativität der Disjunktion
$a \vee (b \vee c) \equiv (a \vee b) \vee c$	Assoziativität der Disjunktion
$a \vee (b \wedge c) \equiv (a \vee b) \wedge (a \vee c)$	Distributivität der Disjunktion bezüglich der Konjunktion
$a \vee (a \wedge b) \equiv a$	Verschmelzungs-(Absorptions-)gesetz
$\neg(a \vee b) \equiv (\neg a \wedge \neg b)$	Gesetz von *De Morgan*
$a \vee b \equiv \neg a \to b$	

Wichtige Äquivalenzen (Forts.)

$a \wedge a \equiv a$	Idempotenz der Konjunktion
$a \wedge \mathrm{w} \equiv a$	
$a \wedge \mathrm{f} \equiv \mathrm{f}$	
$a \wedge b \equiv b \wedge a$	Kommutativität der Konjunktion
$a \wedge (b \wedge c) \equiv (a \wedge b) \wedge c$	Assoziativität der Konjunktion
$a \wedge (b \vee c) \equiv (a \wedge b) \vee (a \wedge c)$	Distributivität der Konjunktion bezüglich der Disjunktion
$a \wedge (a \vee b) \equiv a$	Verschmelzungs-(Absorptions-)gesetz
$\neg(a \wedge b) \equiv (\neg a \vee \neg b)$	Gesetz von *De Morgan*
$a \wedge b \equiv \neg(a \to \neg b)$	
$a \to b \equiv \neg a \vee b$	
$a \to b \equiv \neg(a \wedge \neg b)$	

Wichtige Äquivalenzen

$a \to (b \to c) \equiv b \to (a \to c)$	Vertauschungsgesetz für Prämissen
$(a \leftrightarrow b) \equiv (b \leftrightarrow a)$	Kommutativität der Äquivalenz
$(a \leftrightarrow (b \leftrightarrow c)) \equiv ((a \leftrightarrow b) \leftrightarrow c)$	Assoziativität der Äquivalenz
$(a \leftrightarrow b) \equiv (a \wedge b) \vee (\neg a \wedge \neg b)$	
$(a \leftrightarrow b) \equiv (\neg a \vee b) \wedge (a \vee \neg b)$	
$(a \leftrightarrow b) \equiv (a \to b) \wedge (b \to a)$	

Wichtige Tautologien

$a \to a$	
$a \vee \neg a$	Gesetz vom ausgeschlossenen Dritten tertium non datur
$a \to \neg\neg a$	
$\neg\neg a \to a$	
$a \wedge \neg a \to b$	e falso quodlibet
$a \wedge b \to a$	
$a \wedge b \to b$	
$a \to a \vee b$	Verdünnungsgesetz

Wichtige Tautologien (Forts.)

$a \wedge (a \to b) \to b$	modus ponens (Abtrennungsgesetz)
$(a \to b) \wedge \neg b \to \neg a$	modus tollens (Widerlegungsgesetz)
$(a \to b) \to (\neg b \to \neg a)$	schwache Kontraposition
$(a \to b) \wedge (b \to c) \to (a \to c)$	Transitivitätsgesetz
$a \to (b \to a)$	verum e quodlibet
$(a \to (b \to c)) \to ((a \to b) \to (a \to c))$	Transitivität der Implikation
$(\neg a \to \neg b) \to (b \to a)$	starke Kontraposition

References I

[1] Berger, Krieger, Mahr: "Grundlagen der elektronischen Datenverarbeitung", Skriptum

[2] Blieberger et al.: "Informatik", Springer, 3. Auflage, 1996

CABS: Grundlagen

Computer-Architekturen und Betriebssysteme
Teilbereich: Digitaltechnik

9. Februar 2026

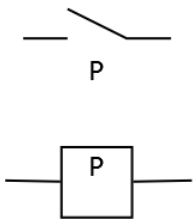
Diese Folien basieren auf dem Skriptum [**?**]

- In elektrischen Schaltkreisen dienen Schalter zum Öffnen oder Schließen von Kontakten.
- Ein Schalter kann *offen* oder *geschlossen* sein:
  - Kein Strom: **0**
  - Strom fließt: **L** [1]
- Die Boole'schen Größen $\{0, 1\}$ werden **Schaltwerte** genannt.
- In der Praxis existiert Toleranzbereich zwischen Spannungsbereichen für jeweilige Schaltwerte.

[1] Bem:L für Leiter

Schalter und Schaltfunktion

- Abbildung rechts:
  - Oben: Schalter P. Strom fließt nur dann, wenn der Schalter geschlossen ist
  - Unten: Schaltplan zu Schalter P

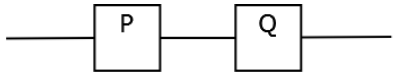


- Schalter werden mit Großbuchstaben $P, Q, R, \ldots$ bezeichnet
- **Schaltfunktion:** Ergebnis (Schaltwert) einer Schaltung in Abhängigkeit von den Schalterzuständen (offen, geschlossen)

Schaltungen

- Zuordnung von aussagenlogischen Variablen zu Schalterzuständen
  - p ... Schalter P geschlossen
  - q ... Schalter Q geschlossen

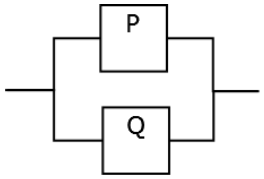
Serienschaltung



- Strom fließt in der Serienschaltung wenn P **und** Q geschlossen sind
- Entsprechender logischer Ausdruck $p \wedge q$

Schaltungen

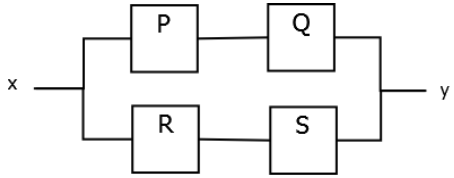
Parallelschaltung



- Strom fließt in der Parallelschaltung wenn P **oder** Q geschlossen sind
- Entsprechender logischer Ausdruck $p \vee q$

Schaltungen

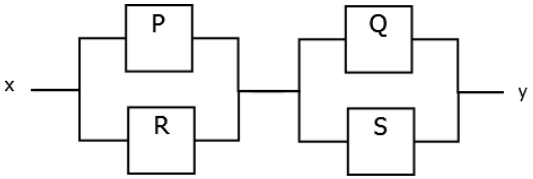
Beispiel: kombinierte Schaltung: $(p \wedge q) \vee (r \wedge s)$



- Anfangs- und Endpunkt der Schaltung werden mit x und y bezeichnet
- Die Schaltfunktion $y = f(p, q, r, s)$ lautet $f(p, q, r, s) = (p \wedge q) \vee (r \wedge s)$

Schaltungen

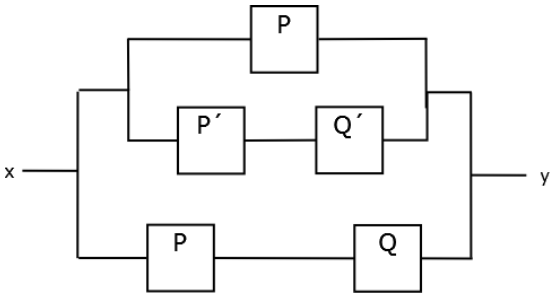
Beispiel: kombinierte Schaltung: $(p \vee r) \wedge (q \vee s)$



- Die Schaltfunktion $y = f(p, q, r, s)$ lautet $f(p, q, r, s) = (p \vee r) \wedge (q \vee s)$

Schaltungen

- Schalter können auch abhängig voneinander wirken
- Derart gekoppelte Schalter können gleichzeitig geschlossen oder geöffnet werden
- Dies entspricht dem mehrfachen Auftreten von Variablen in aussagenlogischen Formeln
- In der Abbildung bezeichnet P' und Q' sind jeweils offen wenn P und Q geschlossen sind (und umgekehrt)
- Schaltfunktion: $f(p, q) = (p \vee (\neg p \wedge \neg q)) \vee (p \wedge q)$



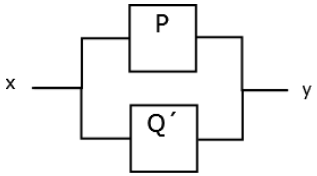
p	q	$f(p, q)$
1	1	1
1	0	1
0	1	0
0	0	1

Schaltungen

Aufgabe 1.1

Vereinfachen Sie den Schaltplan zur gegebenen Schaltfunktion anhand der Methoden der Aussagenlogik.

- In der Schaltung fließt immer Strom, wenn nicht gleichzeitig P offen und Q geschlossen sind.
- Vereinfachte Schaltfunktion: $f(p, q) = p \vee \neg q$



Zusammenhang Aussagenlogik zu elektrischen Schaltkreisen

Aussagenlogik	**Elektrischer Schaltkreis**
Aussagenvariable	Schalter
Aussagenlogischer Ausdruck	Schaltkreis
wahr (w)	Schalter geschlossen (L, 1)
falsch (f)	Schalter geöffnet (0)
Wahrheitsverlauf	Schaltfunktion
Konjunktion	Serienschaltung
Disjunktion	Parallelschaltung
Negation	konträrer Schalterzustand

Aufgabe

Aufgabe 1.2

Ein Komitee, bestehend aus drei Mitgliedern, möchte eine geheime Mehrheitswahl durchführen. Dabei erhält jedes Mitglied einen Schalter mit dem er seine Entscheidung festlegt. Es ist eine Schaltung zu konstruieren, bei der am Ende eine Lampe genau dann aufleuchtet, wenn der Wahlvorschlag angenommen wird (d.h. eine Mehrheit der Mitglieder für "ja" gestimmt hat).

Aufgabe 1.3

In einer Lagerhalle befindet sich in jeder der vier Wände ein Eingangstor mit einem daneben liegendem Lichtschalter. Durch Betätigung eines beliebigen Schalters soll der Zustand der Beleuchtung geändert werden können. Wie müssen diese Schalter miteinander verbunden werden?

CABS: Elektronische Schaltkreise

Computer-Architekturen und Betriebssysteme
Teilbereich: Digitaltechnik

9. Februar 2026

Diese Folien basieren auf dem Skriptum [**?**]

Elektronische Schaltkreise

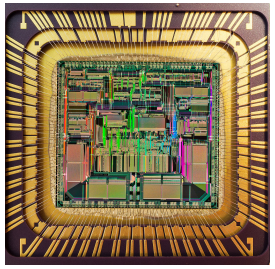
- Realisierung von Schaltungstypen als *Halbleiterbausteine*
- Ein Gatter bildet eine logische Operation auf der (mikro-)elektronischen Ebene ab
- **Schaltnetz:** Anordnung (Verbindung) mehrerer Gatter
- Ein Schaltnetz hat im Allgemeinen mehrere **Eingänge**: **Schaltvariable**
- Die Werte an den Ausgängen des Schaltkreises sind von den Schaltvariablen abhängig

Digitaltechnik

- Bei der Betrachtung von (mikro-)elektronischen Schaltkreisen können die physikalischen Eigenschaften (Spannungsschwankungen, Widerstände, Verzögerungszeiten, Einschwingvorgänge) vernachlässigt werden.
- ⇒ **Digitaltechnik:**
  - digitale Signale sind *diskret*[1] in der Zeit und hinsichtlich ihres Wertebereiches.
  - Zeitdiskret: Takt(frequenz) [2]
  - Wertediskret: Boole'sche Werte 0 und L (= **Binäre Digitaltechnik**)

[1] im Gegensatz zu kontinuierlich
[2] Sequentielle Schaltalgebra: ab Mitte des Semesters

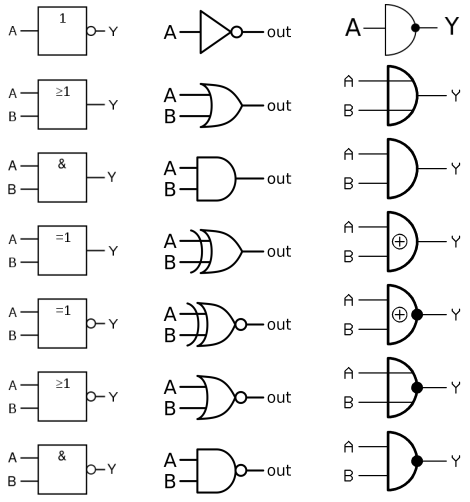
Digitaltechnik



- Die Hauptkomponenten der digitalen Schaltungstechnik sind:
  - Logikgatter: AND, OR, NAND, NOR, XOR, XNOR, ...
  - Kondensatoren als flüchtige Speicher
  - Darauf basierend: komplexere Bauelemente wie Zähler, Flip-Flops
  - Aus diesen Bestandteilen werden schließlich *Mikrocontroller* und *Prozessoren* gebaut
- Hohe Integrationsdichte im Vergleich zu analogen Schaltungen
  - Moderne Mikroprozessoren bestehen aus Milliarden an Transistoren
- Entwicklerwerkzeuge (Computer-aided engineering): VHDL, Verilog
- Hohe Flexibilität, geringe Kosten

Gatter

Bezeichnung	logische Funktion	Gatter (DIN/IEC)	Gatter (US/EN)	DIN (DE, 1976)
Inverter	$y = \neg a$			
OR-Gatter	$y = a \vee b$			
AND-Gatter	$y = a \wedge b$			
XOR-Gatter	$y = a \veebar b$			
XNOR-Gatter	$y = a \leftrightarrow b$			
NOR-Gatter	$y = a \downarrow b$			
NAND-Gatter	$y = a \uparrow b$			

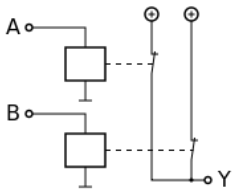


Zusammenhang Aussagenlogik zu elektronischen Schaltkreisen

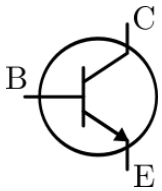
Aussagenlogik	Elektronischer Schaltkreis
Aussagenvariable	Leitung
Aussagenlogischer Ausdruck	Gatter, bzw. Schaltnetz
wahr (w)	Spannungszustand
falsch (f)	Spannungszustand
Wahrheitsverlauf	Schaltfunktion
Negation	Inverter
Konjunktion	AND-Gatter
Disjunktion	OR-Gatter
Antivalenz	XOR-Gatter
Äquivalenz	XNOR-Gatter
Peirce-Operator	NOR-Gatter
Sheffer-Operator	NAND-Gatter

Physikalische Implementierung von Logik-Gattern

- Man könnte alle Gatter durch Schalter (Relais) und Inverter aufbauen
- Abbildung rechts: NAND-Gatter in Relais-Logik
- In der Praxis jedoch Umsetzung mittels Halbleiterbausteinen



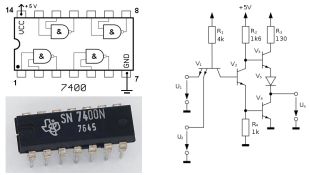
Beispiel: **Transistor:**



- Halbleiterbaustein mit Anschlüssen: Basis (B), Kollektor (C), Emitter (E)
- Kein Strom durch B: Strecke C–E gesperrt (∞ großer Widerstand)
- Strom durch B: C–E wird leitfähig

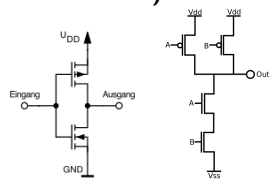
Physikalische Implementierung von Logik-Gattern

- Es existieren verschiedene Technologien zur Umsetzung von Logik-Bausteinen
- **TTL (Transistor-Transistor-Logik):**
  - Heute nur mehr geringe Bedeutung
  - Vorteil: unempfindlich gegenüber elektrostatischen Entladungen
  - Nachteil: hohe Leistungsaufnahme



Links: TTL NAND Baustein
Rechts: TTL NAND Schaltplan

- **CMOS (Complementary Metal Oxide Semiconductor):**
  - Heutzutage meistgenutzte Logikfamilie



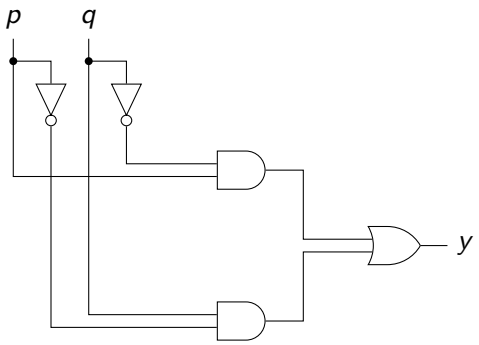
Links: CMOS-Inverter, rechts: CMOS-NAND

Anwendungen

- Logik-Bausteine, integrierte Schaltkreise:
  - 74xx Familie: verschiedene Varianten, TTL, CMOS
- **Application-Specific Integrated Circuit (ASIC):**
  - auch Custom-Chips genannt
  - z.B. in PCs
- **Field Programmable Gate Array (FPGA):**
  - Konkrete Schaltungsstruktur kann frei "programmiert" werden (z.B. mit VHDL)
  - Kann beliebig oft neu programmiert werden

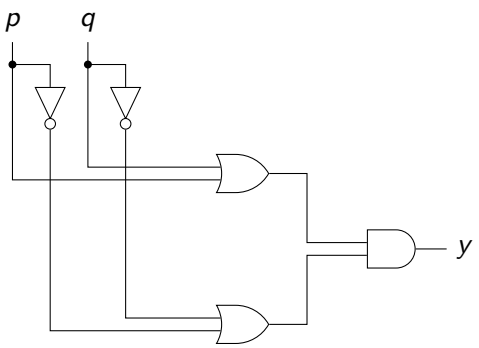
Beispiel: XOR-Gatter

- Schaltplan für ein XOR-Gatter mit AND, OR und Inverter in SOP Darstellung
- $p \veebar q = (p \wedge \neg q) \vee (\neg p \wedge q)$



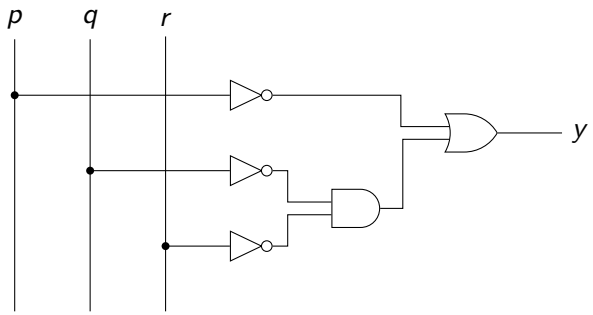
Beispiel: XOR-Gatter

- Schaltplan für ein XOR-Gatter mit AND, OR und Inverter in POS Darstellung
- $p \veebar q = (p \vee q) \wedge (\neg p \vee \neg q)$



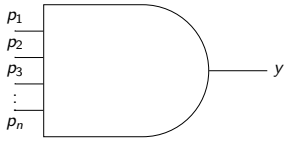
Beispiel: Schaltplan

Beispiel: Schaltplan für $\neg p \vee (\neg q \wedge \neg r)$

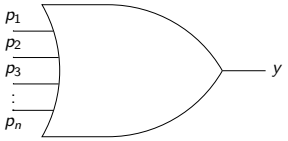


Schaltkurzzeichen

- Werden mehrere Eingänge $p_1, p_2, p_3, \ldots, p_n$ miteinander verknüpft, so kann man die Gatter zusammenfassen
- **Schaltkurzzeichen können nur bei assoziativen Operationen angewandt werden!**
- **Anmerkung:** NAND und NOR sind *nicht* assoziativ!



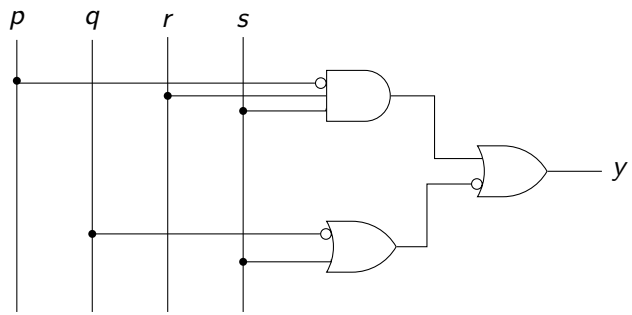
Schaltfunktion: $y = p_1 \wedge p_2 \wedge p_3 \wedge \ldots \wedge p_n$



Schaltfunktion: $y = p_1 \vee p_2 \vee p_3 \vee \ldots \vee p_n$

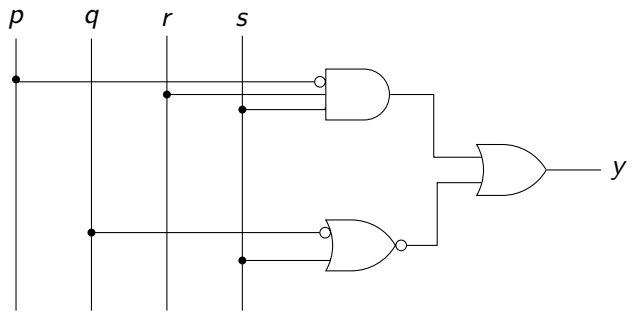
Beispiel: Schaltplan für $(\neg p \wedge r \wedge s) \vee \neg(\neg q \vee s)$

- Statt Invertern werden hier einfach Kreise vor die Eingänge der Gatter gesetzt
- Ebenso werden Schaltkurzzeichen verwendet
- Beides ist beim Abzählen der Gatter zu berücksichtigen (Inverter an Eingängen jedoch insgesamt nur einmal!)



Beispiel: Schaltplan für $(\neg p \wedge r \wedge s) \vee \neg(\neg q \vee s)$

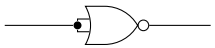
- Bei der zusätzlichen Verwendung von NOR-Gattern ist folgende Umformung möglich: $(\neg p \wedge r \wedge s) \vee (\neg q \downarrow s)$



- Die Schaltung enthält zwei UND-, ein ODER- und ein NOR-Gatter, sowie zwei Inverter.

Inverter mit NAND/NOR

- Inverter können durch NAND, bzw. NOR-Gatter umgesetzt werden
- $$\neg a \equiv a \downarrow a \equiv a \uparrow a$$
- Umsetzung im Schaltplan indem eine Leitung verzweigt und an beide Eingänge eines NAND- bzw. NOR-Gatters geführt wird



Aufgabe 2.1

Gegeben sei die Funktion $y = (p \wedge q) \vee (\neg r \wedge \neg(p \wedge q))$.

1. Zeichnen Sie einen Schaltplan, der unmittelbar der gegebenen Funktion entspricht.
2. Zeichnen Sie einen Schaltplan mit möglichst wenigen Gattern (AND, OR, NOT) zur gegebenen Funktion.
3. Zeichnen Sie einen Schaltplan mit einer *minimalen* Anzahl an (beliebigen) Gattern
4. Zeichnen Sie einen weiteren Schaltplan, indem ausschließlich NAND-Gatter verwendet werden.

Aufgabe 2.2

Gegeben Sei das Ergebnis einer Schaltfunktion:

0101 0111

1. Ermitteln Sie eine Schaltung mit *minimaler* Anzahl an verwendeten Gattern.
2. Ermitteln Sie eine weitere Schaltung, die nur NAND-Gatter verwendet.
3. Ermitteln Sie eine weitere Schaltung, die nur NOR-Gatter verwendet.

Aufgabe 2.3

Gegeben Sei das Ergebnis einer Schaltfunktion:

0000 0101 0011 0011

1. Ermitteln Sie eine Schaltung mit *minimaler* Anzahl an verwendeten Gattern.
2. Ermitteln Sie eine weitere Schaltung, die nur NAND-Gatter verwendet.

Aufgabe 2.4

Gegeben Sei das Ergebnis einer Funktion:

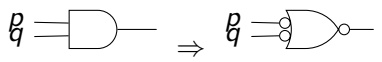
1011 1001 0010 0001

1. Ermitteln Sie eine Schaltung mit minimaler Anzahl an verwendeten Gattern.
2. Ermitteln Sie eine weitere Schaltung, die nur NAND-Gatter verwendet.

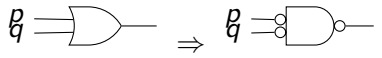
De Morgan mit Gattern

Die De Morgan'schen Gesetze werden in der Gatter-Darstellung besonders anschaulich:

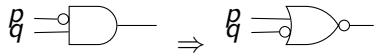
$p \wedge q \quad \Rightarrow \quad \neg(\neg p \vee \neg q)$



$p \vee q \quad \Rightarrow \quad \neg(\neg p \wedge \neg q)$



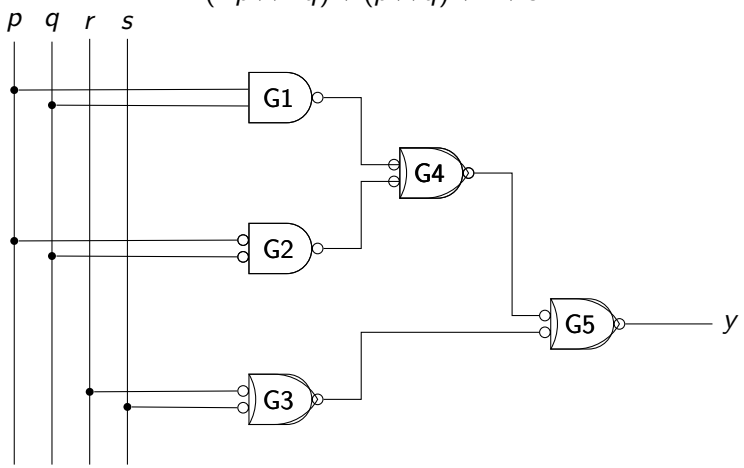
$\neg p \wedge q \quad \Rightarrow \quad \neg(p \vee \neg q)$



Beobachtung: Bei der Umwandlung von (N)AND in (N)OR Gatter

Wir betrachten die Schaltfunktion

$$(\neg p \wedge \neg q) \vee (p \wedge q) \vee r \vee s$$



Das Oder-Gatter G3 wird durch ein NAND-Gatter mit zwei invertierten Eingängen ersetzt.Analog wird das Oder-Gatter G4 durch ein NAND-Gatter mit zwei invertierten Eingängen ersetzt. Analog wird mit

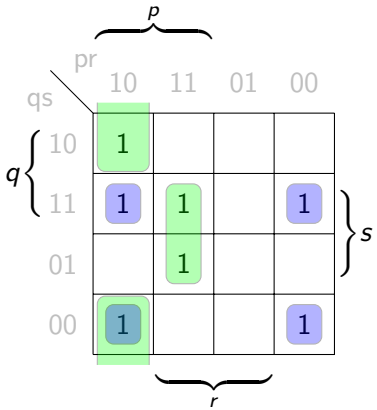
Aufgabe 2.4

Gegeben Sei das Ergebnis einer Funktion:

1011 1001 0010 0001

1. Ermitteln Sie eine Schaltung mit minimaler Anzahl an verwendeten Gattern.
2. Ermitteln Sie eine weitere Schaltung, die nur NAND-Gatter verwendet (ebenso mit einer minimalen Anzahl an Gattern).

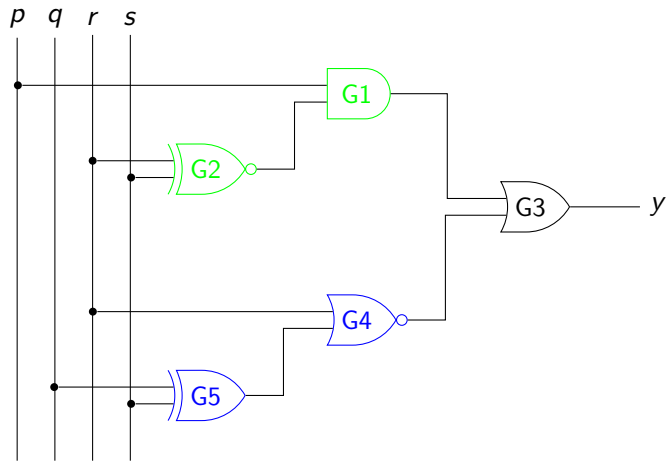
Wir betrachten den Verlauf der Schaltfunktion 1011 1001 0010 0001



Wir erhalten die Formel: $p \wedge (r \leftrightarrow s) \vee \neg r \wedge (q \leftrightarrow s)$ oder noch besser

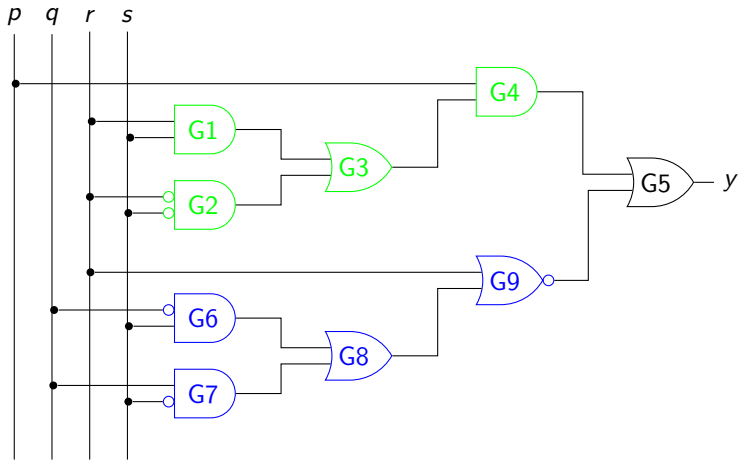
$$p \wedge (r \leftrightarrow s) \vee (r \downarrow (q \veebar s))$$

Formel: $p \wedge (r \leftrightarrow s) \vee (r \downarrow (q \veebar s))$



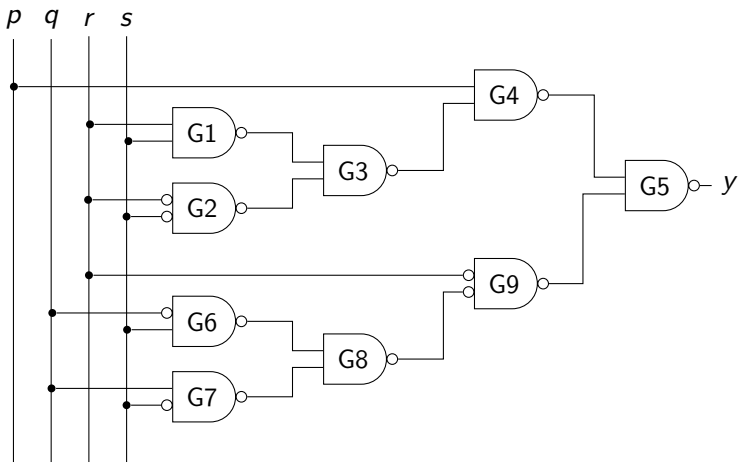
Die Schaltung benötigt insgesamt 5 Gatter.

In Hinblick auf die NAND-Umformung eliminieren wir aus dem vorigen Schaltplan nun die XOR- und XNOR Gatter, indem wir sie durch AND/OR/NEGATION ausdrücken.

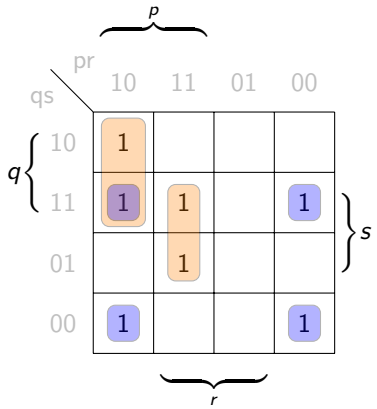


$$p \wedge (r \wedge s \vee \neg r \wedge \neg s) \vee (r \downarrow (q \wedge \neg s \vee \neg q \wedge s))$$

Durch Umwandlung der "Oder-Typ" Gatter in "Und-Typ" Gatter erhalten wir eine reine NAND-Schaltung mit insgesamt **13 Gattern**

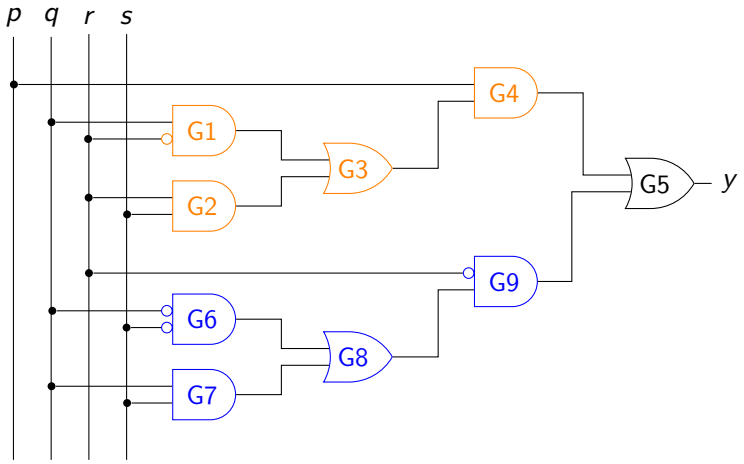


- Diese Lösung verwendete als Ausgangspunkt die 5-Gatter Lösung mit beliebigen Gatter-Typen.
- In diesem Beispiel führt jedoch ein anderer Ausgangspunkt zu einer geringfügig besseren Lösung (wie im Folgenden dargestellt).



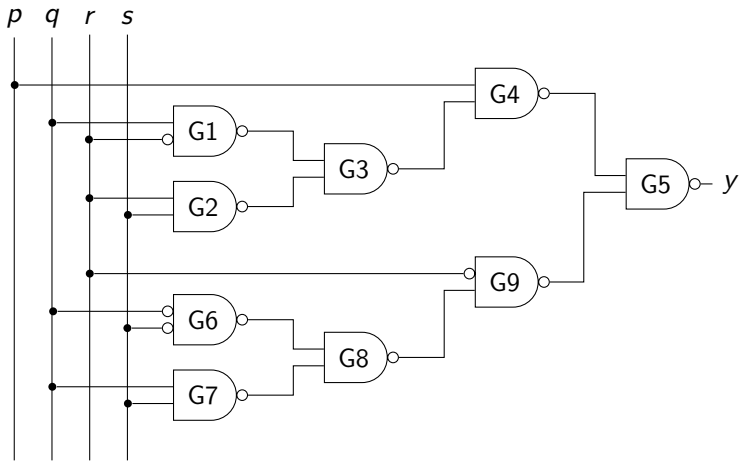
Wir erhalten die Formel: $p \wedge (q \wedge \neg r \vee r \wedge s) \vee \neg r \wedge (q \leftrightarrow s)$

Schaltplan mit ausschließlich AND/OR/Inverter:



$p \wedge (q \wedge \neg r \vee r \wedge s) \vee \neg r \wedge (q \leftrightarrow s)$

Die reine NAND-Schaltung hat nun 9 NAND-Gatter und Inverter für q, r und s, also ingesamt **12 Gatter**!



CABS: Addier-Schaltungen

Computer-Architekturen und Betriebssysteme
Teilbereich: Digitaltechnik

9. Februar 2026

Diese Folien basieren auf dem Skriptum [**?**]

Anmerkung: Die Animationen sehen Sie am Besten durch Weiterschalten der Folien im Vollbildmodus!

Halbaddierwerk

- Wir entwickeln zunächst eine Schaltung zur Addition zweier Binärzahlen p und q
- Die Rechenregeln lauten:

$$0 + 0 = 0$$
$$0 + 1 = 1$$
$$1 + 0 = 1$$
$$1 + 1 = 10$$

- Bei der Addition von $1 + 1$ entsteht also ein **Übertrag**, der gesondert behandelt werden muss.

Halbaddierwerk

- Für die Summe s und den Übertrag u erhalten wir die folgenden Wahrheitstabellen:

p	q	s
1	1	0
1	0	1
0	1	1
0	0	0

p	q	u
1	1	1
1	0	0
0	1	0
0	0	0

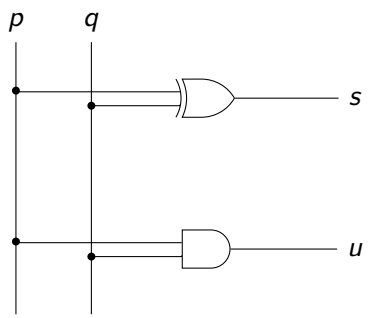
- Daraus ergeben sich für die Summe s und den Übertrag u folgende Schaltfunktionen

$$s \leftrightarrow p \veebar q$$

$$u \leftrightarrow p \wedge q$$

Halbaddierwerk

Schaltplan:



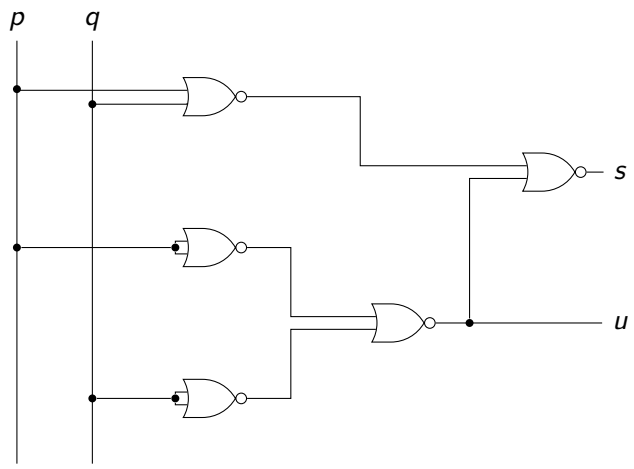
Halbaddierwerk

Beispiel: Umsetzung eines Halbaddierwerk mit ausschließlich NOR-Gattern:

$$
\begin{aligned}
s &\leftrightarrow p \veebar q \\
&\leftrightarrow \neg((p \wedge q) \vee (\neg p \wedge \neg q)) \\
&\leftrightarrow \neg(\neg(\neg p \vee \neg q) \vee (\neg(p \vee q))) \\
&\leftrightarrow ((p \downarrow p) \downarrow (q \downarrow q)) \downarrow (p \downarrow q)
\end{aligned}
$$

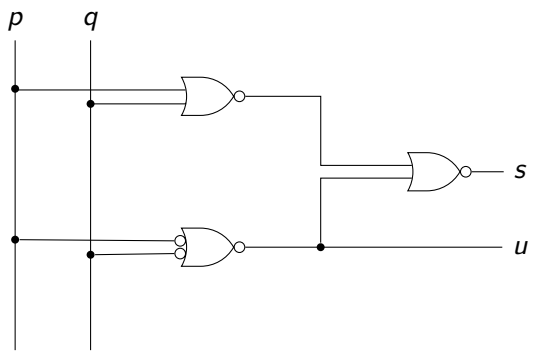
$$
\begin{aligned}
u &\leftrightarrow p \wedge q \\
&\leftrightarrow \neg(\neg p \vee \neg q) \\
&\leftrightarrow (p \downarrow p) \downarrow (q \downarrow q)
\end{aligned}
$$

Halbaddierwerk mit ausschließlich NOR-Gattern



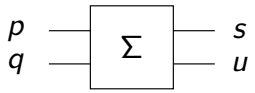
Halbaddierwerk mit ausschließlich NOR-Gattern

Schaltplan mit ausschließlich NOR-Gattern in Kurzschreibweise (mit negierten Eingängen an den NOR-Gattern, statt expliziter Verwendung von NOR-Gattern als Inverter):



Halbaddierwerk

Die Schaltung wird als **Halbaddierwerk** bezeichnet, und durch folgendes Symbol dargestellt:



Frage

Warum heißt diese Schaltung *Halbaddierwerk*?

Antwort: bei der Addition von zwei Stellen einer Binärzahl muss auch der Übertrag der vorangegangenen Stelle mit berücksichtigt werden. Dies ist hier noch nicht der Fall, weshalb die Schaltung als *Halbaddierwerk* bezeichnet wird.

Addition mehrstelliger Binärzahlen

- Wir betrachten die Addition zweiter Binärzahlen $p = 01011$ und $q = 01110$ mit jeweils 5 Stellen
- Die einzelnen Stellen von p bezeichnen wir mit (von rechts nach links) mit p_0, p_1, p_2 usw., oder kurz p_i mit $i = 0, 1, \ldots$; (analog für q)
- Die Addition der beiden Zahlen verläuft gemäß:

Index i	4	3	2	1	0
p_i	0	1	0	1	1
q_i	0	1	1	1	0
Übertrag u_i	0	1	1	1	0
Ergebnis s_i	1	1	0	0	1

$s_0 = p_0 + q_0 = 1 + 0 = 1,\ u_0 = 0\ s_1 = p_1 + q_1 + u_0 = 1 + 1 + 0 = 0,$
$u_1 = 1\ s_2 = p_2 + q_2 + u_1 = 0 + 1 + 1 = 0,\ u_2 = 1$
$s_3 = p_3 + q_3 + u_2 = 1 + 1 + 1 = 1,\ u_3 = 1$
$s_4 = p_4 + q_4 + u_3 = 0 + 0 + 1 = 1,\ u_4 = 0$

Volladdierwerk

- Um mehrstellige Binärzahlen addieren zu können muss also jeweils der Übertrag u_{i-1} des vorhergehenden Additionsschrittes beachtet werden.
- Somit ergibt sich folgende Wahrheitstabelle:

p_i	q_i	u_{i-1}	u_i	s_i
1	1	1	1	1
1	1	0	1	0
1	0	1	1	0
1	0	0	0	1
0	1	1	1	0
0	1	0	0	1
0	0	1	0	1
0	0	0	0	0

Volladdierwerk: s_i

Für die Summe s_i ergibt sich folgendes KV-Diagramm:

q_i \ $p_i u_{i-1}$	10	11	01	00
q_i = 1	0	1	0	1
q_i = 0	1	0	1	0

und somit

$$s_i = (p_i \wedge q_i \wedge u_{i-1}) \vee (p_i \wedge \neg q_i \wedge \neg u_{i-1}) \vee (\neg p_i \wedge q_i \wedge \neg u_{i-1}) \vee (\neg p_i \wedge \neg q_i \wedge u_{i-1})$$

Volladdierwerk: s_i

Wir formen den Ausdruck nun so um, dass möglichst viele Äquivalenzen und Antivalenzen auftreten, um XNOR- und XOR-Gatter verwenden zu können.

$$\begin{aligned} s_i &= ((p_i \wedge q_i) \vee (\neg p_i \wedge \neg q_i)) \wedge u_{i-1} \vee ((\neg p_i \wedge q_i) \vee (p_i \wedge \neg q_i) \wedge \neg u_{i-1} && (1) \\ &= ((p_i \leftrightarrow q_i) \wedge u_{i-1}) \vee ((p_i \veebar q_i) \wedge \neg u_{i-1}) && (2) \\ &= (p_i \leftrightarrow q_i) \leftrightarrow u_{i-1} && (3) \\ &= (p_i \veebar q_i) \veebar u_{i-1} && (4) \end{aligned}$$

Das Ergebnis aus Zeile 3 ermöglicht uns die Schaltung mit zwei XNOR-Gattern zu realisieren, Zeile 4 liefert die Lösung mit zwei XOR-Gattern.

Volladdierwerk: u_i

Für den Übertrag u_i ergibt sich folgendes KV-Diagramm:

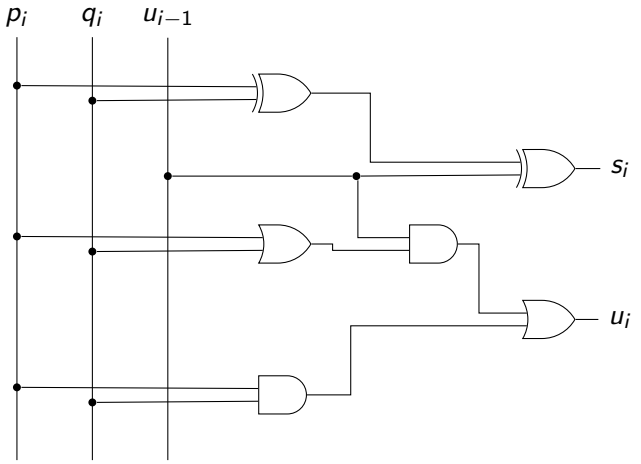
$q_i \backslash p_i u_{i-1}$	10	11	01	00
q_i = 1	1	1	1	0
q_i = 0	0	1	0	0

und somit

$$u_i = (p_i \wedge q_i) \vee ((p_i \vee q_i) \wedge u_{i-1})$$

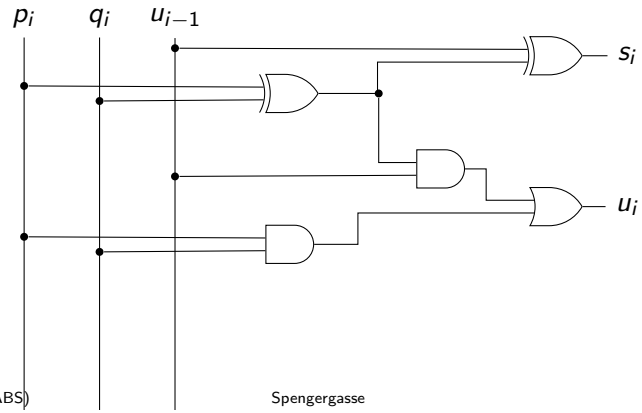
Volladdierwerk

Unter Verwendung von XOR-Gattern für die Summe ergibt sich folgendes Schaltbild:



Volladdierwerk

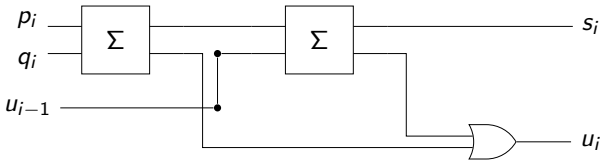
Weitere Verbesserungen lassen sich erzielen, wenn man für u_i die Formel $u_i = (p_i \wedge q_i) \vee ((p_i \veebar q_i) \wedge u_{i-1})$ verwendet. $(p_i \veebar q_i)$ ist nun gemeinsamer Teilausdruck sowohl von s_i als auch u_i und muss in der Schaltung nur einmal realisiert werden.



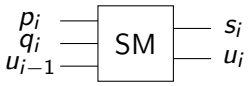
Volladdierwerk

Alternative Überlegung:

- Zwei Ziffern addieren und zur Summe die dritte Ziffer hinzuzählen
- Übertrag kann durch OR-Gatter abgebildet werden, da nie an beiden Halbaddierern gleichzeitig ein Übertrag auftritt



Die Schaltung wird als **Volladdierwerk** bezeichnet, und durch folgendes Symbol dargestellt:



CABS: Flip-Flops

Computer-Architekturen und Betriebssysteme
Teilbereich: Digitaltechnik

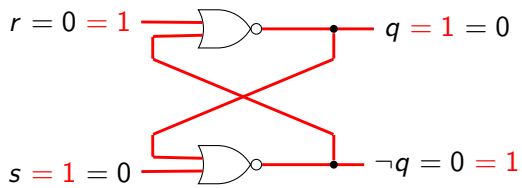
9. Februar 2026

Diese Folien basieren auf dem Skriptum [1]

Einleitung

- Bisher zeitliche Abläufe nicht berücksichtigt.
- Durch neue Eingangswerte ändern sich sofort alle Ergebnisse.
- **Ziel:** Information soll in Bauelementen gespeichert werden
- Umsetzung durch Rückkopplung von NOR-Gattern

RS-Flip-Flop

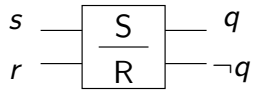


An den Ausgängen steht sowohl q als auch dessen Negation zur Verfügung.

Wird am s-Eingang (s für "set") kurzzeitig 1 angelegt, so wird der Ausgang q=1.
Aufgrund der Rückkopplung bleibt 1 am Ausgang auch dann erhalten, wenn der s-Eingang wieder 0 wird.Erst wenn am r-Eingang 1 angelegt wird, dann erscheint am Ausgang q wieder 0.

RS-Flip-Flop

Somit kann ein Bit gespeichert werden. Das RS-Flip-Flop (RS-FF) wird durch folgendes Schaltsymbol dargestellt:

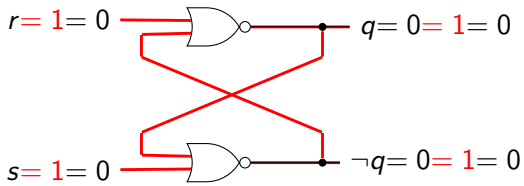


Wir betrachten nun die Zustandsänderungen anhand einer Tabelle:

s_n	0	0	0	0	1	1	1	1
r_n	0	0	1	1	0	0	1	1
q_n	0	1	0	1	0	1	0	1
q_{n+1}	0	1	0	0	1	1	0	0

RS-Flip-Flop

Wir betrachten nun die Situation $s = r = 1$:**Nun schalten wir** $s = r = 0$ **(Speicherzustand):**



Beide Ausgänge schalten auf 0. Dies ist an sich ein stabiler Zustand, führt jedoch zu instabilen/undefinierten Zuständen bei den folgenden Eingangswerten $r = s = 0$ (Speicherzustand).Übergangszustand: Beide Ausgänge q und $\neg q$ werden nun 1. Um die zeitliche Abhängigkeit zu verdeutlichen ist dieser Zustand nicht vollständig eingezeichnet, die rückgekoppelten Leitungen werden (erst) unmittelbar darauffolgend rot.Nun liegt an beiden NOR-Gattern eine 1 an. Die NOR-Gatter haben (gerade) noch nicht geschalttet.Nach Schalten der NOR-Gatter liegt an

RS-Flip-Flop

Der Zustand $r = s = 1$ ist also beim RS-FF in jedem Fall zu vermeiden!

s_n	0	0	0	0	1	1	**1**	**1**
r_n	0	0	1	1	0	0	**1**	**1**
q_n	0	1	0	1	0	1	0	1
q_{n+1}	0	1	0	0	1	1	0	0

RS-Flip-Flop

$r_n \backslash s_n q_n$	10	11	01	00
r_n = 1	0	0	0	0
0	1	1	1	0

s_n q_n

Die **charakteristische Gleichung** des RS-FFs lautet somit

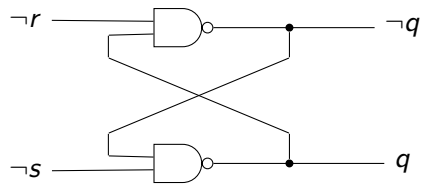
$$q_{n+1} = \neg r_n \wedge (s_n \vee q_n)$$

mit der Nebenbedingung

$$r_n \wedge s_n = 0$$

RS-FF mit NAND-Gattern

- Das RS-FF kann auch mittels NAND-Gattern realisiert werden.
- Die Eingänge sind hierbei $\neg s$, bzw. $\neg r$.
- Der "verbotene Zustand" ist diesfalls $\neg r = \neg s = 0$.

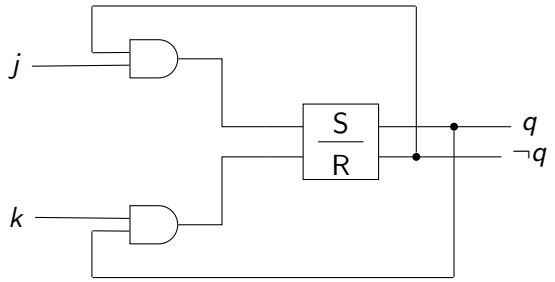


Zusammenfassung RS-FF

- Das RS-FF kann 1 Bit Information speichern.
- Der s-Eingang setzt den Inhalt auf 1, der r-Eingang setzt auf 0 (zurück).
- Sind beide Eingänge $r = s = 0$, so wird die gesetzte Information gespeichert.
- $r = s = 1$ führt dazu, dass beide Ausgänge 0 werden.
- Dies hat (insbesondere bei $r = s = 0$) die Auswirkung, dass das RS-FF zu schwingen beginnt, oder in einen undefinierten Zustand übergeht (in Abhängigkeit davon, welches NOR-Gatter schneller schaltet)
- Das JK-Flip-Flop löst dieses Problem.

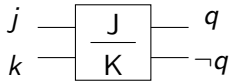
JK-Flip-Flop

Mithilfe zweier UND-Gatter kann verhindert werden, dass an beiden Eingängen gleichzeitig 1 anliegt:



JK-Flip-Flop

Das resultierende Gatter wird JK-Flip-Flop genannt, und durch folgendes Symbol dargestellt:



Die Wahrheitstabelle lautet:

j_n	0	0	0	0	1	1	1	1
k_n	0	0	1	1	0	0	1	1
q_n	0	1	0	1	0	1	0	1
q_{n+1}	0	1	0	0	1	1	1	0

JK-Flip-Flop

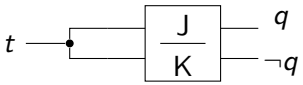
Wir erhalten mittels KV-Diagramm

$k_n \backslash j_n q_n$	10	11	01	00
1	1	0	0	0
0	1	1	1	0

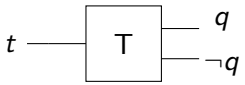
die charakteristische Gleichung:

$$q_{n+1} = (j_n \wedge \neg q_n) \vee (\neg k_n \wedge q_n)$$

T-Flip-Flop



Das T-Flip-Flop (von engl. "toggle"-flip-flop, ist eigentlich ein JK-Flip-Flop dessen Eingänge kurzgeschlossen sind. Mit jeder Eins am Eingang ändert das T-Flip-Flop seinen Zustand (ähnlich einem Schalter). Häufig wird folgende Kurzdarstellung verwendet:



T-Flip-Flop

Die charakteristische Gleichung des T-FFs kann unmittelbar aus der Wahrheitstabelle entnommen werden, und entspricht genau der charakteristischen Gleichung des JK-FFs, wenn für j_n und k_n jeweils t_n eingesetzt wird:

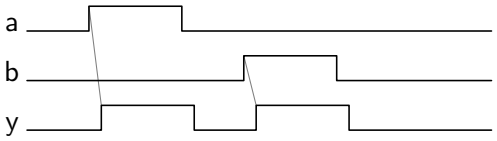
t_n	0	0	1	1
q_n	0	1	0	1
q_{n+1}	0	1	1	0

Die charakteristische Gleichung lautet somit:

$$q_{n+1} = (t_n \wedge \neg q_n) \vee (\neg t_n \wedge q_n) = t_n \veebar q_n$$

Schaltverhalten von Gattern

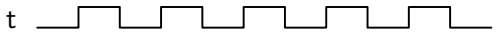
- Bisher haben wir Schaltungen betrachtet, die immer *unmittelbar* nach Anliegen der Eingangswerte schalten.
- "Unmittelbar" bedeutet jedoch in der Praxis: mit geringer Verzögerungszeit.
- Im *Impulsdiagramm* (engl. timing-diagram) wird der zeitliche Ablauf (horizontale Achse) von $y = a \vee b$ dargestellt



- Die Verzögerung von y (bezüglich a oder b) beträgt einige Nanosekunden.

Taktsignal

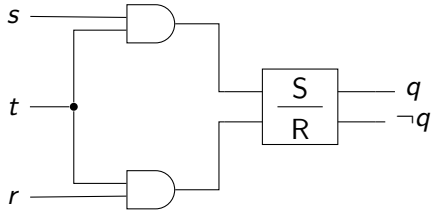
- In einem *sequentiellen* Netzwerk ändern sich die Zustände zu diskreten Zeitpunkten.
- Diese Zeitpunkte sind durch das *Taktsignal* vorgegeben.
- Als Taktgenerator wird ein Schwingquarz als Oszillator verwendet



- Die Zeit zwischen zwei steigenden Flanken wird mit t_0 bezeichnet
- Die (Takt-)Frequenz ist gegeben durch $f_0 = 1/t_0$

Getaktetes RS-Flip-Flop

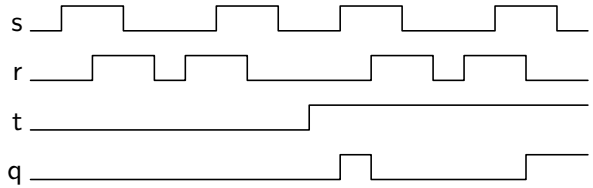
- Durch Vorschalten eines *Takttors* mittels zwei UND-Gattern kann sichergestellt werden, dass das RS-Flip-Flop während eines Taktimpulses schaltet.



- Das getaktete RS-FF wird oft als *RST-FF* bezeichnet.
- Die oben dargestellte Variante des RST-FFs reagiert nur *während* $t = 1$ auf die angelegten Eingangswerte.
- Deshalb wird es auch als *taktzustandsgesteuertes* RST-FF bezeichnet.

Taktzustandsgesteuertes RS-FF

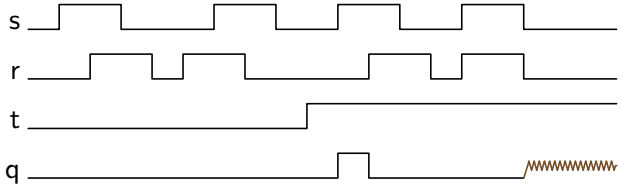
- Wir untersuchen das Verhalten des taktzustandsgesteuertes RS-FFs im Impulsdiagramm



- Solange $t = 0$ (Low) ändern die angelegten Werte an r und s nichts (Q bleibt unverändert)
- Bei $t = 1$ werden alle Änderungen sofort übernommen.
- Zwei Mal wird der "verbotene Zustand" $r = s = 1$ eingenommen, allerdings ist der Folgezustand jeweils nicht ($r = s = 0$) wodurch sich keine weiteren Probleme ergeben.

Taktzustandsgesteuertes RS-FF

- Wir betrachten hier nochmals die Situation des Schwingungszustandes bei $r = s = 0$ nach $r = s = 1$ (mit jeweils $t = 1$)



Begrifflichkeiten

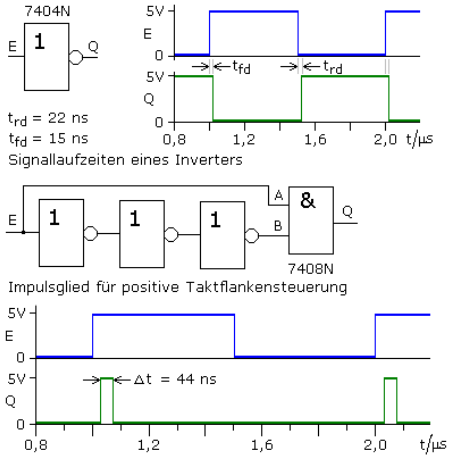
- In der englischsprachigen Literatur wird oftmals zwischen den Begriffen *Latch* und *Flip-Flop* unterschieden:
  - **Latch:** Nicht-getaktetes, bzw. taktzustandsgesteuertes Bauelement
  - **Flip-Flop:** Takt**flanken** gesteuertes Bauelement
- In der deutschsprachigen Literatur wird meist nur der Begriff Flip-Flop verwendet.
- Wir verwenden ebenso ausschließlich den Begriff Flip-Flop.

Bei Taktsignalen der folgenden Form verhalten sich takt-zustands- und flanken- gesteuerte Flip-Flops gleich.

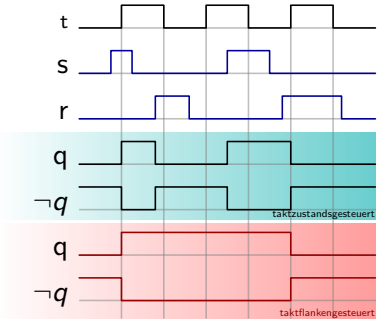
Anmerkung: Im Folgenden wollen wir ein derartiges Taktsignal voraussetzen.

Taktsignal

Schaltung zur Generierung eines kurzen Nadelimpulses aufgrund des Taktsignals:



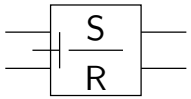
Vergleich zwischen *taktzustandsgesteuertem* und *taktflankengesteuertem* RS-FF

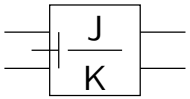


Quelle: verändert von `http://commons.wikimedia.org/wiki/File:SR_FF_timing_diagram.png` sowie `http://www.texample.net/tikz/examples/more-tikz-timing-examples/`

RST- und JKT Flip-Flop

Taktgesteuerte RS- und JK- FFs werden auch als *RST- und JKT Flip-Flop*s bezeichnet. Die Schaltsymbole sind:

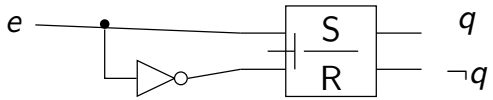




- Ein Netzwerk indem sämtliche FFs taktgesteuert sind nennt man **synchrones** Netzwerk.
- Der Takt wird am mittleren Eingang angelegt.
- Auf das Einzeichnen der Taktleitungen wird der Übersicht halber jedoch oft verzichtet.

D Flip-Flop

Ein taktgesteuertes RS-FF, an dessen r-Eingang der verneinte s-Eingang liegt, gestattet es, das Eingangssignal um einen Taktimpuls zu verzögern:



Setzt man in der charakteristischen Gleichung des RS-FFs

$$q_{n+1} = \neg r_n \wedge (s_n \vee q_n)$$

für $s_n = e_n$ und für $r_n = \neg e_n$, so erhalten wir nach Anwendung des Verschmelzungsgesetzes

$$q_{n+1} = e_n.$$

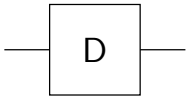
D Flip-Flop

Die charakteristische Gleichung des D-FFs lautet

$$q_{n+1} = e_n.$$

Das bedeutet, dass am Ausgang der um das Taktintervall t_0 verzögerte Eingang anliegt.

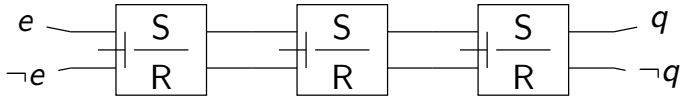
Kurzschaltsymbol:



Das D-Flip-Flop wird oft *Delay*-Flip-Flop oder *Data*–Flip-Flop genannt.

Schieberegister

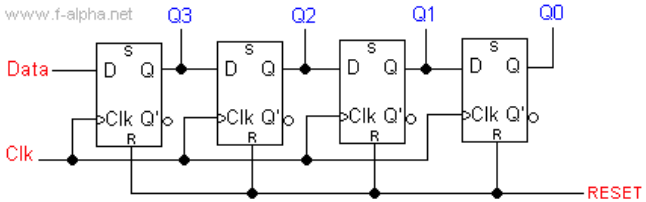
- Wird eine Folge solcher Verzögerungselemente hintereinander geschaltet, so erhält man eine Verzögerungskette, die als **Schieberegister** bezeichnet wird.
- Die Taktleitungen werden zur besseren Übersicht nur angedeutet.



- Da jedes FF auch den negierten Ausgang zur Verfügung stellt, erübrigt sich die Verwendung der Inverter.

- Flip-Flops müssen in gültigen Anfangszustand versetzt werden.
- Dafür existieren weitere Eingänge:
  - Startwert 0: "Clear" (CLR), manchmal auch (bei D-FFs) "Reset" (R)
  - Startwert 1: "Preset" (PR)

Beispiel: Schieberegister



Schieberegister

- Ein aus n FFs bestehendes Schieberegister kann eine n-stellige Binärzahl speichern.
- Die einzelnen Binärziffern werden der Reihe nach an den Eingang angelegt und mit jedem Taktimpuls um eine Stelle verschoben.
- Am Ausgang stehen die Ziffern dann zur weiteren Verarbeitung zur Verfügung.
- Verwendung zum Speichern von Operanden und Ergebnissen arithmetischer Operationen.

Literaturübersicht I

[1] **Berger, Krieger, Mahr: "Grundlagen der elektronischen Datenverarbeitung", Skriptum**

[2] Jürgen Reichardt: "Digitaltechnik", De Gruyter Oldenbourg, 4. Auflage, 2017

[3] Blieberger et al.: "Informatik", Springer, 3. Auflage, 1996

[4] Patterson, Hennessy: "Computer Organization & Design", Second Edition, Morgan Kaufmann, 1998

[5] Wikipedia (Englisch): `https://en.wikipedia.org/`

[6] Wikipedia (Deutsch): `https://de.wikipedia.org/`

CABS: Zähler-Schaltungen

Computer-Architekturen und Betriebssysteme
Teilbereich: Digitaltechnik

9. Februar 2026

Diese Folien basieren auf dem Skriptum [**?**]

Zähler

Zähler besitzen in der Digitaltechnik vielfältige Anwendungen, wie z.B.:

- Digitaluhren, digitale Anzeigen
- Zählen von Ereignissen
- Program-Counter (Programmzähler in Mikroprozessoren)
- Steuerung von Automaten

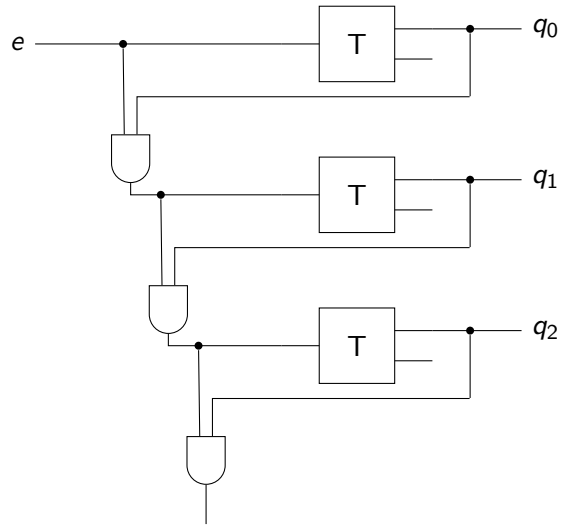
In der Praxis werden überwiegend **Synchronzähler** verwendet. Alle Flip-Flops werden dabei durch einen gemeinsamen Takt angesteuert.

Anmerkung: In den Schaltskizzen werden die Taktleitungen der Einfachkeit halber weggelassen.

Zähler

- In der folgenden Abbildung (nächste Folie) werden n T-FFs so zusammengeschaltet, dass eine n-stellige Binärzahl gespeichert werden kann.
- Bei jedem Impuls am Eingang e wird die Zahl um 1 erhöht.
- Wertebereich: $0 \ldots 2^n - 1$
- Die Ausgänge $q_0, q_1, q_2, \ldots$ der T-FFs stellen die Ziffern des Zählerstandes dar.
- Das erste T-FF ändert seinen Zustand mit jedem Impuls, alle übrigen ändern ihren Zustand dann, wenn das vorhergehende von 1 auf 0 schaltet.

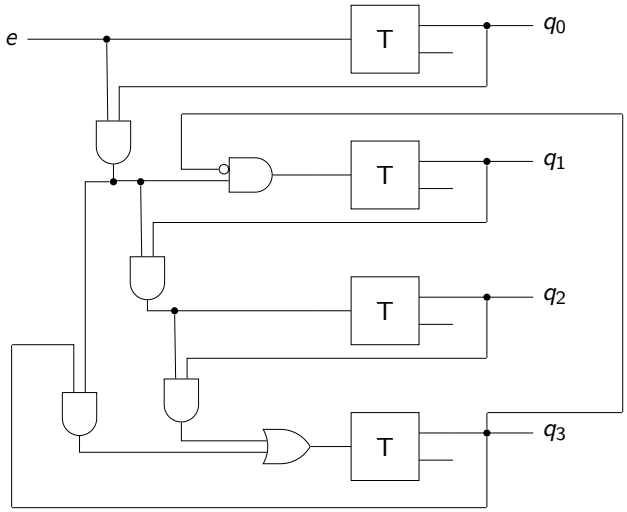
3-Bit Zähler



4-Bit Dezimalzähler

- Durch 4 T-FFs kann eine Hexadezimalziffer gespeichert werden.
- Für *Dezimalzähler* muss nach Zahl 9 ($= (1001)_2$) wieder zu 0 gewechselt werden.
- Somit: 2. und 3. T-FF sollen *nicht* schalten, jedoch das 1. und 4.
- Das erste T-FF schaltet beim nächsten Impuls sowieso auf 0 $\Rightarrow$ **es muss verhindert werden, dass das 2. T-FF schaltet!**
- Das 3. T-FF würde nicht schalten.
- Das 4. T-FF würde ebenso nicht schalten $\Rightarrow$ **es muss erzwungen werden, dass das 4. T-FF schaltet!**

4-Bit Dezimalzähler



Aufgabe 06.01

Konstruieren Sie eine möglichst einfache elektronische Schaltung, die die Phasen einer Verkehrsampel ansteuert:

Grün – Gelb – Rot – Rot/Gelb

Aufgabe 06.02

Konstruieren Sie unter Verwendung von T-Flip-Flops einen Zähler, der

a) bis 5,

b) bis 13

zählt, und dann wieder auf 0 zurückspringt.

Konstruktion von Zählern

- Sei q die Zahl nach der der Zähler wieder auf 0 zurückgesetzt werden soll, und $q_n, q_{n-1}, \ldots, q_1, q_0$ ihre Binärdarstellung.
- Die T-FFs zur Stelle i ($i = 0 \ldots n$) bezeichnen wir als i-tes Flip-Flop.
- Es existiert mindestens ein $q_i = 0$ (alle $q_i \neq 0$ ist trivial)
- Wir betrachten nun die Stelle $q_k = 0$ mit minimalem k, also die am weitesten rechts stehende 0. (Danach folgen entweder 1en oder nichts mehr)

Beispiel: Begleitendes Beispiel: Zähler von 0 bis 43

1	0	1	**0**	1	1
q_5	q_4	q_3	$\mathbf{q_2}$	q_1	q_0

$q_2 = 0 \Rightarrow k = 2$

Konstruktion von Zählern

- Das FF k mit $q_k = 0$ würde beim nächsten Taktimpuls auf 1 wechseln.
  - **Das wird verhindert, indem alle Ausgänge der T-FFs links von k die Wert 1 haben mit UND verknüpft werden.**
  - **Der Ausgang dieses UND-Gatters wird negiert an den Eingang eines weiteren UND-Gatters vor dem FF von q_k geführt.**
  - Der andere Eingang dieses UND-Gatters ist die ursprüngliche Leitung.
  - Dies bewirkt das Sperren des Umschaltens von q_k ab dem Zeitpunkt wenn die links davon stehenden "1"-Flip-Flops 1 geworden sind.
- Weiters muss erzwungen werden, dass die "1"-FFs links von k bei Erreichen der Zahl q auf 0 zurückgesetzt werden.
  - Dies soll also nur stattfinden wenn für alle Binärstellen mit $q_i = 1$ von q alle T-FFs auf 1 stehen.
  - Die UND-Verknüpfung aller "1"-FFs (links von k) und des Ereignisimpulses wird an ODER-Gatter angelegt.
  - Diese werden an die Eingänge der "1"-FFs (links von k) angelegt.

Konstruktion von Zählern

Beispiel: Begleitendes Beispiel: Zähler von 0 bis 43

Dezimalzahl	q_5	q_4	q_3	q_2	q_1	q_0	
0_{10}	0	0	0	0	0	0	
43_{10}	1	0	1	0	1	1	letzter Zähl-Wert
44_{10}	1	0	1	1	0	0	erster Wert der **nicht** erreicht werden soll
	+	ok	+	-	ok	ok	notwendige Änderungen

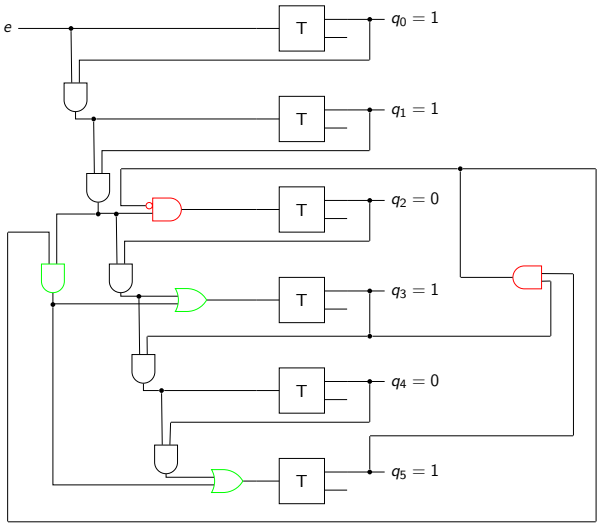
Hierbei bedeutet:

+ **erzwingen**, dass das Flip-Flop schaltet (obwohl es eigentlich nicht schalten würde)

- **verhindern**, dass das Flip-Flop schaltet.

ok das FF schaltet, und dies ist auch gewünscht *oder* FF schaltet nicht, und dies ist gewünscht

Zähler: 0 bis 43



Abwärts-Zähler

Wir betrachten in der Tabelle ebenso die Werte der negierten Ausgänge der T-FFs:

q_3, q_2, q_1, q_0	$k_{(10)}$	$\neg q_3, \neg q_2, \neg q_1, \neg q_0$	$m_{(10)}$
0000	0	1111	15
0001	1	1110	14
0010	2	1101	13
0011	3	1100	12
0100	4	1011	11
⋮		⋮	
1001	9	0110	6
1010	10	0101	5
1011	11	0100	4
⋮		⋮	
1111	15	0000	0

- Der Dezimalzähler zählt an den verneinten Ausgängen von 15 bis 6 *herunter*.
- Die verneinten Ausgänge stellen binär das Einerkomplement dar.

Aufgabe 06.03

Konstruieren Sie unter Verwendung von T-Flip-Flops einen Zähler, der

a) von 0 bis 26,
b) von 0 bis 49,
c) von 0 bis 53

zählt, und dann wieder auf den Anfangswert zurückspringt.

Aufgabe 06.04

Konstruieren Sie unter Verwendung von T-Flip-Flops einen Zähler, der

a) von 31 bis 9,
b) von 63 bis 17

zählt, und dann wieder auf den Anfangswert zurückspringt.

Aufgabe 06.05

Konstruieren Sie unter Verwendung von T-Flip-Flops einen Zähler, der

a) von 5 bis 49,
b) von 19 bis 53,
c) von 53 bis 22

zählt, und dann wieder auf den Anfangswert zurückspringt.

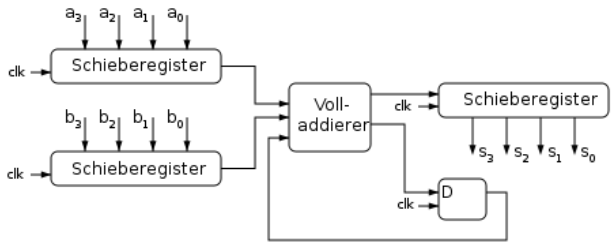
TINF: Weitere Komponenten

Andreas M. Chwatal

Technische Informatik
Digitaltechnik

16. März 2021

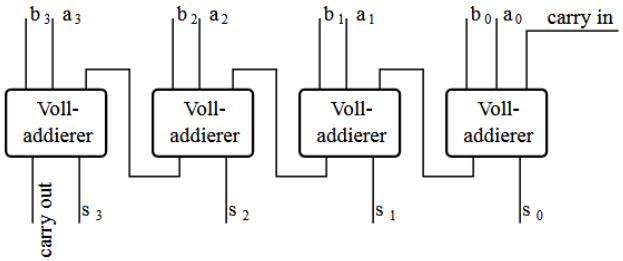
Serienaddierwerk



Quelle: [6]

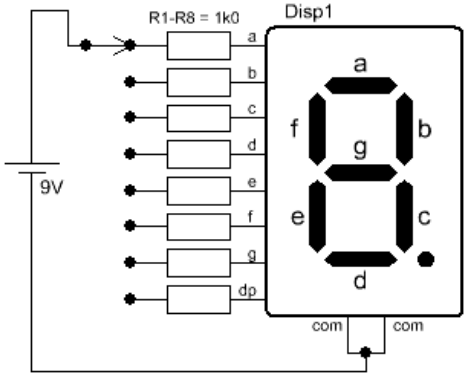
Paralleladdierwerk

Ripple-Carry-Addierer:

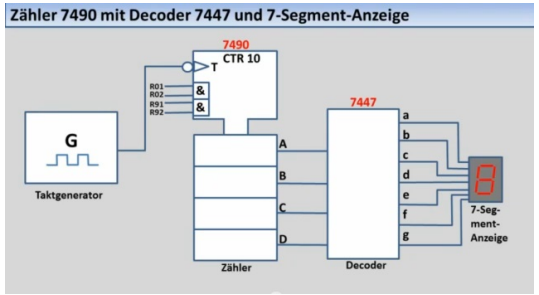


Quelle: [6]

7-Segment-Anzeige



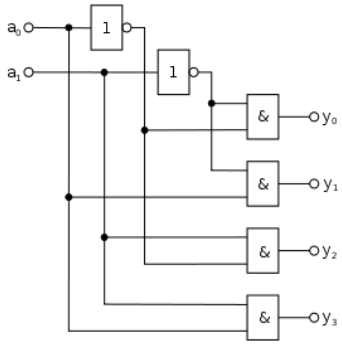
7-Segment-Anzeige



Quelle: `https://et-tutorials.de/7353/zahler-frequenzteiler-decoder/`

Decoder

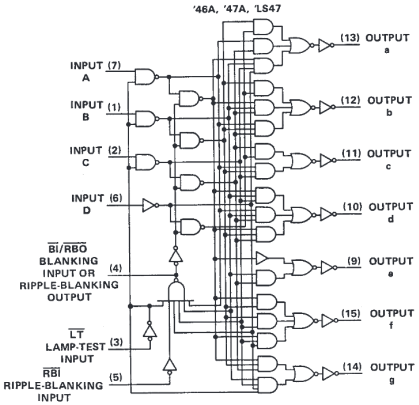
Wert A	Eingang a_1	a_0	Ausgang y_3	y_2	y_1	y_0
0	0	0	0	0	0	1
1	0	1	0	0	1	0
2	1	0	0	1	0	0
3	1	1	1	0	0	0



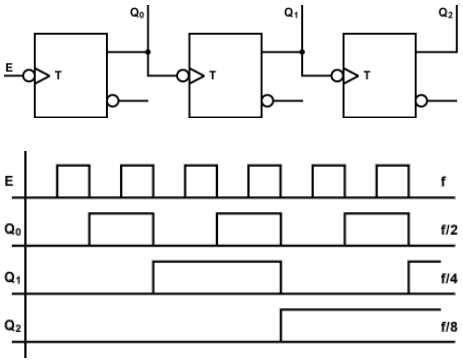
Quelle: [6]

7-Segment-Decoder

decimal	x3	x2	x1	x0	UC	UL	UR	CE	LL	LC	LR
0	0	0	0	0	1	1	1	0	1	1	1
1	0	0	0	1	0	0	1	0	0	0	1
2	0	0	1	0	1	0	1	1	1	1	0
3	0	0	1	1	1	0	1	1	0	1	1
4	0	1	0	0	0	1	1	1	0	0	1
5	0	1	0	1	1	1	0	1	0	1	1
6	0	1	1	0	1	1	0	1	1	1	1
7	0	1	1	1	1	0	1	0	0	0	1
8	1	0	0	0	1	1	1	1	1	1	1
9	1	0	0	1	1	1	1	1	0	0	1
10	1	0	1	0	X	X	X	X	X	X	X
11	1	0	1	1	X	X	X	X	X	X	X
12	1	1	0	0	X	X	X	X	X	X	X
13	1	1	0	1	X	X	X	X	X	X	X
14	1	1	1	0	X	X	X	X	X	X	X
15	1	1	1	1	X	X	X	X	X	X	X



Frequenzteiler



Quelle: [7]

Literaturübersicht I

[1] **Berger, Krieger, Mahr: "Grundlagen der elektronischen Datenverarbeitung", Skriptum**

[2] Jürgen Reichardt: "Digitaltechnik", De Gruyter Oldenbourg, 4. Auflage, 2017

[3] Blieberger et al.: "Informatik", Springer, 3. Auflage, 1996

[4] Patterson, Hennessy: "Computer Organization & Design", Second Edition, Morgan Kaufmann, 1998

[5] Wikipedia (Englisch): `https://en.wikipedia.org/`

[6] Wikipedia (Deutsch): `https://de.wikipedia.org/`

[7] Elektronik-Kompendium: `http://www.elektronik-kompendium.de/`

CABS: Festkommadarstellung

Computer-Architekturen und Betriebssysteme
Teilbereich: Digitaltechnik

9. Februar 2026

Diese Folien basieren auf dem Skriptum [**?**]

Festkommadarstellung

Definition (Festkommadarstellung)

Bei der *Festkommadarstellung* wird eine bestimmte Anzahl an Bits als Vorkomma-Stellen, sowie eine bestimmte Anzahl an Bits als Nachkomma-Stellen verwendet.

Beispiel: Darstellung mit Vorzeichenbit (VZ) und $g = 4$ Vorkommastellen und $d = 3$ Nachkommastellen

VZ	g	g	g	g	d	d	d

Somit lautet die Zahl $+12.75$:

0	1	1	0	0	1	1	0

Anmerkung: Die Festkommadarstellung wird heute kaum noch verwendet. Wichtig ist jedoch die sogenannte *Festkommaarithmetik* für die Arithmetik mit vorzeichenbehafteten ganzen Zahlen (also $d = 0$).

Festkommadarstellung

Wir betrachten also die Situation ($d = 0$):

VZ	g	g	g	g	g	g	g

Beispiel:

$$\begin{array}{rr} 000\ldots1011 & 11_{10} \\ -000\ldots1101 & -13_{10} \\ \hline 111\ldots1110 & -2_{10} \end{array}$$

Mit n Bit können 2^n Zahlen dargestellt werden.

1	1	1	1	1	1	1	1
1	1	1	1	1	1	1	0
				⋮			
1	0	0	0	0	0	0	0
0	1	1	1	1	1	1	1
				⋮			
0	0	0	0	0	0	0	1
0	0	0	0	0	0	0	0

Festkommadarstellung

- Vorzeichenbit ergibt sich *automatisch* aufgrund der Anforderung, dass alle Rechenoperationen funktionieren müssen.
- Bei n Bit können 2^n Zahlen dargestellt werden ($0 \ldots 2^n - 1$)
- Erste nicht darstellbare Zahl hat Wert 2^n

Folgerung: $-x$ ergibt sich durch $2^n - x$

Beispiel: $n = 8$:

$$\begin{array}{rl} 100000000 & (2^8 = 256) \\ -00000011 & (-3) \\ \hline 11111101 & (-3) \end{array}$$

Mit n Bit können 2^n Zahlen dargestellt werden.

1	1	1	1	1	1	1	1
1	1	1	1	1	1	1	0
				⋮			
1	0	0	0	0	0	0	0
0	1	1	1	1	1	1	1
				⋮			
0	0	0	0	0	0	0	1
0	0	0	0	0	0	0	0

Festkommadarstellung

- Vorzeichenbit ergibt sich *automatisch* aufgrund der Anforderung, dass alle Rechenoperationen funktionieren müssen.
- Bei n Bit können 2^n Zahlen dargestellt werden $(0 \dots 2^n - 1)$
- Erste nicht darstellbare Zahl hat Wert 2^n

Zweierkomplement

Da 2^n nicht darstellbar ist, verwenden wir $2^n - x = 2^n - 1 - x + 1$ und erhalten somit die negative Darstellung einer Zahl durch Invertierung aller Bits und anschließende Addition von 1.

Mit n Bit können 2^n Zahlen dargestellt werden.

1	1	1	1	1	1	1	1
1	1	1	1	1	1	1	0
				⋮			
1	0	0	0	0	0	0	0
0	1	1	1	1	1	1	1
				⋮			
0	0	0	0	0	0	0	1
0	0	0	0	0	0	0	0

Einer- und Zweier-Komplement

Definition (Einerkomplement)

Negative Zahl entsteht aus Invertierung aller Bits einer positiven Zahl

Nachteile:

- Null ist doppelt dargestellt
- Probleme beim Durchschreiten der Null bei einer Rechenoperation

Definition (Zweierkomplement)

Negative Zahl entsteht aus Invertierung aller Bits einer positiven Zahl und anschließender Addition von Eins.

Bemerkung:

- Vorherrschende Art der Zahlendarstellung in Computern

Zweierkomplement

- Mit n Bit können 2^n Zahlen dargestellt werden.
- 2^n ist erste nicht darstellbare Zahl.
- Wertebereich Zweierkomplement: $[-2^{n-1}, 2^{n-1} - 1]$

Zahl im Dualsystem									Potenz	Dezimal	2er-K. (Dez.)
1	0	0	0	0	0	0	0	0	2^n	nicht darstellbar	
	1	1	1	1	1	1	1	1	$2^n - 1$	255	**-1**
	1	1	1	1	1	1	1	0	$2^n - 2$	254	**-2**
					⋮					⋮	⋮
	1	0	0	0	0	0	0	0	2^{n-1}	128	**-128**
	0	1	1	1	1	1	1	1	$2^{n-1} - 1$	127	**127**
					⋮						
	0	0	0	0	0	0	0	1	1	1	**1**
	0	0	0	0	0	0	0	0	0	0	**0**

Multiplikation

- *Beispiel:* $(-13_{10}) \cdot (-27_{10})$
- Im Zweierkomplement:

$$(2^n - 13) \cdot (2^n - 27) = 2^{2n} - 40 \cdot 2^n + 351$$

- Algorithmus zur Multiplikation:
  - Zwei positive Zahlen: normale Multiplikation
  - Zwei negative Zahlen: von jeder Zahl 2er-K. berechnen und dann multiplizieren
  - Eine negative Zahl: 2er-K. von negativer Zahl; vom Ergebnis wieder 2er-K. berechnen

CABS: Gleitkommadarstellung

Computer-Architekturen und Betriebssysteme
Teilbereich: Digitaltechnik

9. Februar 2026

Diese Folien basieren auf dem Skriptum [**?**]

Gleitkommadarstellung

Definition (Gleitkommadarstellung)

Darstellungsform einer Zahl mittels einer festen Anzahl an *signifikanten* Stellen, gemeinsam mit einem Skalierungsfaktor.

Ziffernfolge: **Mantisse**
Skalierungsfaktor: **Exponent**

Sei m die Anzahl der Mantissenstellen, und e die Anzahl der Exponenten-Stellen.

Somit wird eine Zahl Z zur Basis B dargestellt durch:

$$Z = m \cdot B^e$$

Gleitkommadarstellung

- Vorzeichen der Mantisse V_m
- Mantissenstelle m
- Vorzeichen des Exponenten V_e
- Exponentenstelle e

V_m	m	m	m	m	m	m	m	m	m	m	V_e	e	e	e	e

Gleitkommadarstellung

Konkrete Umsetzung:

- Expontent: Festkommadarstellung **(d.h. Zweierkomplement)**
- Mantisse: Vorzeichenbit und (positive) Zahl

Varianten:

1.														
V_m	m	m	m	m	m	m	m	m	V_e	e	e	e	e

2. | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| V_m | V_e | e | e | e | e | m | m | m | m | m | m | m | m |

3. | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| V_e | e | e | e | e | V_m | m | m | m | m | m | m | m | m |

- Format 2 (bzw. Varianten davon) wird tatsächlich verwendet.
  - Zahlreiche Optimierungen der Reihenfolge um effizientere Berechnungen zu ermöglichen.
- **Im Unterricht: Format 1**

Normalisierte Gleitkommadarstellung

Die Darstellung einer Zahl $Z = m \cdot B^e$ ist *nicht eindeutig*!

Beispiel: $45.67 = 4567 \cdot 10^{-2} = 0.4567 \cdot 10^2$

Deshalb verwendet man die *eindeutige* normalisierte Gleitkommadarstellung!

Definition (Normalisierte Gleitkommadarstellung)

Für Mantisse m muss gelten:

- $-1 < m \leq -0.1$, oder
- $0.1 \leq m < 1$.

- Tatsächlich gespeichert wird nur der Nachkommaanteil der Mantisse!
- Eine Ausnahme stellt die Darstellung der Zahl 0 dar (mehr dazu später)

Normalisierte Gleitkommadarstellung

Beispiel: $123.47_{10} = 0.12347_{10} \cdot 10^3$

Beispiel: $11010.11_2 = 0.1101011_2 \cdot 2^5$

Normalisierte Gleitkommadarstellung

Beispiel: Darstellung von 5.75 mit $B = 2, m = 5$ und Wortlänge $l = 10$ $\Rightarrow e = 3$

$$5.75 = 101.11_2 = 0.10111_2 \cdot 2^3$$

V_m	m	m	m	m	m	V_e	e	e	e
0	1	0	1	1	1	0	0	1	1

Beispiel: Darstellung von -5.75

V_m	m	m	m	m	m	V_e	e	e	e
1	1	0	1	1	1	0	0	1	1

Darstellbare Zahlen

Genauigkeit:

- Angenommen 5 Stellen im Zehnersystem
- Genauigkeit bei Zahlen in der Größenordnung
  - einer Million: auf 100 genau
  - 1000: $^1/_{10}$ Genauigkeit
- Folgerung: Abstand benachbarter Zahlen variiert. Kleinere darstellbare Zahlen liegen enger beisammen als größere.

Darstellbarer Bereich:

- Größter Exponent: $B^e - 1$
- Größte Mantisse (im Dualsystem):

$$0.11111111 \Rightarrow \frac{B^m - 1}{B^m} = 1 - \frac{1}{B^m}$$

Darstellbare Zahlen

Bereich der darstellbaren Zahlen:

$$\left[-\frac{B^m-1}{B^m}B^{B^e-1}, \frac{B^m-1}{B^m}B^{B^e-1}\right]$$

Abstand zweier benachbarter Zahlen $\frac{B^k}{B^m}$ mit $-B^e-1 \leq k \leq B^e-1$

Mögliche Optimierungen

Charakteristik:

- Anwendung bei kurzen Wortlängen (heute nicht mehr verwendet).
- Größere Exponenten werden häufiger benötigt als kleine Exponenten.
- Lösung: Zu Exponent e wird Konstante k addiert
- k wird *Charakteristik*, *Bias* oder *Exzess* genannt

Einsparung eines Mantissenbits:

- Erstes Bit ist (gemäß Normalisierung) immer 1!
- $\Rightarrow$ kann somit weggelassen (eingespart) werden!
- Weitere Mantissenstelle wird verfügbar $\Rightarrow$ höhere Genauigkeit
- Jedoch **Problem bei der Darstellung von 0!**
- Lösung: kleinste darstellbare Zahl wird als 0 definiert.

Konversion Dezimal $\rightarrow$ Gleitkomma

Beispiel: Zahlenformat: $l = 16, m = 9, e = 5$
Darstellung von 3.14_{10} als Gleitkommazahl:

$3.14_{10} = 11.0010001_2 = 0.110010001 \cdot 2^2$

V_m	m	m	m	m	m	m	m	m	m	V_e	e	e	e	e	e
0	1	1	0	0	1	0	0	0	1	0	0	0	0	1	0

Konversion Gleitkomma $\rightarrow$ Dezimal

Beispiel: Konvertieren Sie zurück ins Dezimalsystem:

V_m	m	m	m	m	m	m	m	m	m	V_e	e	e	e	e	e
1	1	1	0	0	0	0	0	0	0	1	1	1	1	0	0

Exponent: $111100_2 \ldots$ negative Zahl
$\Rightarrow$ positive Zahl 000100 $\Rightarrow$ Exponent ist -4.

$-0.110000000 \cdot 2^{-4} = -0.000011 \cdot 2^0$
$= -(1/32 + 1/64) = -(0.03125 + 0.015625) = -0.046875$

Grundlagen

- In der Festkommadarstellung ist Punktrechnung nicht direkt möglich.
  - Begründung: $-x$ wird im Zweierkomplement dargestellt als $2^n - x = 2^n - 1 - \mathbf{x} + \mathbf{1}$
  - Konsequenz beim Multiplizieren:

    $$(-x) \cdot (-y) = x \cdot y,$$

    jedoch würde die direkte Multiplikation als Ergebnis liefern:

    $$(2^n - x) \cdot (2^n - y) = 2^{2n} - 2^n \cdot x - 2^n \cdot y + xy$$

  - Es müssen also Fallunterscheidungen getroffen werden (zunächst Konversion auf positive Zahlen, anschließende Vorzeichenberechnung)
- **Folgerung:** Bei Exponent wird Festkommadarstellung (2-er Komplement) verwendet, da nur Addition und Subtraktion benötigt werden.
- Die Mantisse wird mit explizitem Vorzeichenbit abgespeichert.

Addition/Subtraktion

- Zunächst Anpassung an größeren Exponenten
- Beispiel:

$$\begin{array}{lcl} 0.11 & \cdot & 2^1 \\ 0.11 & \cdot & 2^{-1} \\ \hline \end{array}$$

Anpassung an 2^1:

$$\begin{array}{lcl} 0.11 & \cdot & 2^1 \\ 0.0011 & \cdot & 2^1 \\ \hline 0.1111 & \cdot & 2^1 \end{array}$$

- Gegebenenfalls muss das Ergebnis wieder normalisiert werden!

Multiplikation/Division

- Mantissen werden multipliziert/dividiert.
- Exponenten werden addiert/subtrahiert.
- Bei einer Multiplikation können eventuell weitere Nachkommastellen mit 0 entstehen, z.B. $0.1 \cdot 0.1 = 0.01$
- In diesem Fall muss wieder normalisiert werden, d.h. gegebenenfalls Anpassung des Exponenten (-1/+1)

CABS: Prädikatenlogik

Computer-Architekturen und Betriebssysteme
Teilbereich: Digitaltechnik

9. Februar 2026

Diese Folien basieren auf dem Skriptum [**?**]

Prädikatenlogik

- Die Prädikatenlogik (PL) erweitert die schon bekannte Aussagenlogik.
- Meist wird unter "Logik" die *Prädikatenlogik* verstanden.
- George Boole und Auguste De Morgan bereiteten bereits die PL vor (PL erster Stufe).
- Die PL zweiter Stufe wurde von Gottlob Frege eingeführt, und Bertrand Russel, Alfred North Whitehead weiterentwickelt.
- Wir betrachten nur die PL erster Stufe.

Motivation

Ein klassisches Beispiel:

- Alle Menschen sind sterblich.
- Sokrates ist ein Mensch.

Offensichtlich folgt aus diesen beiden Aussagen:

- Sokrates ist sterblich.

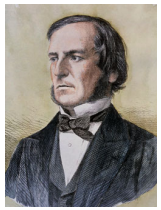
Jedoch: anhand der Aussagenlogik (AL) ist diese Schlussfolgerung nicht möglich! Begründung: Die AL betrachtet nur *atomare* (unteilbare) Aussagen, deren Bedeutung kann jedoch nicht genauer untersucht werden.

Die Prädikatenlogik erweitert die Aussagekraft und Mächtigkeit der AL.

Formale Aussagenlogik und Prädikatenlogik

George Boole (*1815, †1864)

- Englischer Philosoph und Mathematiker
- Begründer der *symbolischen Aussagenlogik*
- Mathematische Behandlung *atomarer Aussagen*, die entweder *wahr* oder *falsch* sein können
- Verknüpfung dieser Aussagen durch logische *Operatoren*



Auguste De Morgan (*1806, †1871)

- Englischer Mathematiker
- Mit-Begründer der *Mathematischen Induktion*
- De Morgansche Gesetzte zur Umformung von *Konjunktion* in *Diskunktion*

Prädikatenlogik

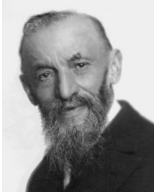
Boole und De Morgan bereiteten bereits die Prädikatenlogik vor. Die Prädikatenlogik zweiter Stufe wurde von Gottlob Frege eingeführt, und Bertrand Russel, Alfred North Whitehead weiterentwickelt.

Gottlob Frege (*1848, †1925)

- Deutscher Philosoph, Logiker und Mathematiker
- Logik als Basis der Mathematik
- Einführung der Quantoren
- Begründer der Prädikatenlogik (parallel zu *Peirce*)

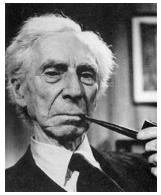
Giuseppe Peano (*1858, †1932)

- Italienischer Mathematiker und Logiker
- Begründer der axiomatischen Mengenlehre
- Formalisierung der vollständigen Induktion



Bertrand Russel (*1872, †1970)

- Englischer Philosoph, Logiker, Mathematiker
- Autor der *Principia Mathematica*



Die Prädikatenlogik zweiter Stufe ermöglichte schließlich die logische Untermauerung der Arithmetik.

Anwendungen

- "Sprache" der Mathematik
- Logikorientierte Programmiersprachen (PROLOG)
- Expertensysteme
- Formalisierung und Modellierung von Problemstellungen (Informatik, Wirtschaft, Industrie, u.v.m.)

Überblick: Elemente der PL

Die PL erweitert die AL um:

- **Subjekte:**
  Beispiel: $x \ldots$ Menschen
- **Prädikate:**
  Beispiel: $S(x) \ldots x$ ist sterblich
- **Funktionen:**
  Beispiel: $f(x)$
- **Quantoren:**
  - Allquantor: $\bigwedge$ (Bedeutung: *für alle...*)
  - Existenzquantor: $\bigvee$ (Bedeutung: *es existiert...*)

Die Bedeutung und Gesetzmäßigkeiten der *Operatoren (Junktoren)* der AL bleiben in der PL unverändert erhalten.

Konventionen und ergänzende Bemerkungen

- **Subjekte**
  - Bezeichnung mit Kleinbuchstaben, z.B. $x, y, z, \ldots$
- **Prädikate**
  - Bezeichnung mit Großbuchstaben
  - Wahl des Buchstabens nach der Bedeutung
- **Funktionen**
  - Bezeichnung mit Kleinbuchstaben, z.B. $f, g, h, \ldots$

Bezüglich der **Quantoren** ist auch folgende Schreibweise gebräuchlich:

- Allquantor: $\forall$
- Existenzquantor: $\exists$

Jedoch: Wir verwenden $\bigwedge$ und $\bigvee$, da diese Symbole ähnlich zu den Junktoren $\wedge$ und $\vee$ sind, und hierbei tatsächlich die gleichen Gesetzmäßigkeiten gelten!

Subjektvariablen (Gegenstandsvariablen)

- Die Subjektvariablen (auch Gegenstandsvariablen) repräsentieren die betrachteten Objekte/Subjekte/Gegenstände.
- Bei der Definition wird der *Subjektbereich* (auch Wertebereich) angegeben.

Beispiel: $x \ldots$ Menschen

Beispiel: $y \ldots \mathbb{N}$ (also y aus den natürlichen Zahlen)

Beispiel: $z \ldots$ Fahrzeuge

Beispiel: $v \ldots$ Knoten eines zusammenhängenden Graphen

Beispiel: $w \ldots \{\mathrm{rot}, \mathrm{gruen}, \mathrm{blau}\}$

Funktionen

- In der PL werden *Funktionen*, im Sinne einer mathematischen Abbildung von einer Menge A in eine Menge B verwendet:

$$f : A \to B$$

- *Beispiel:* Mit $x \ldots \mathbb{N}$ ist eine Abbildung $f : \mathbb{N} \to \mathbb{N}$ beispielsweise definiert durch die Vorschrift $f(x) = x^2$.
- *Beispiel:* Mit $x, y \ldots$ Menschen: $m(x) = y$ sei hierbei eine Funktion m die einem Menschen x seine Mutter y zuordnet.
- Funktionen können auch mehrere Argumente haben: $f(x, y, z)$
- *Beispiel:* n-stellige Funktion: $f(x_1, x_2, \ldots, x_n)$
- *Beispiel:* 3-stellige Funktion: $f(x, y, z) = x + y \cdot z$

Wichtig: Eine Abbildung in die Menge {wahr, falsch} ist **keine Funktion**. (Das wäre ein Prädikat)

Prädikate

- In der Grammatik versteht man unter einem Prädikat die *Satzaussage*.
- In der PL können Prädikate *Aussagen* oder *Beziehungen* zu/von Subjekten (Gegenständen) ausdrücken.
- *Beispiel:*

Prädikat	Aussage	Subjektbereich
$L(x)$	x läuft	$x \ldots$ Menschen
$R(y)$	y ist rund	$y \ldots$ geometrische Figuren
$S(z)$	z ist eine Stadt	$z \ldots$ geografische Objekte

- **Für konkrete Objekte ergibt ein Prädikat entweder wahr oder falsch!**

Prädikate

- Auch Prädikate können *mehrstellig* sein!
- *Beispiel:*

Prädikat	Beziehung (Aussage)
$G(x, y)$	x ist größer als y
$L(x, y, z)$	x liegt zwischen y und z
$M(x, y)$	x ist Mutter von y

- Man beachte, dass $G(x, y)$ ungleich $G(y, x)$!
- *Beispiel:* n-stelliges Prädikat: $P(x_1, x_2, \ldots, x_n)$
- Prädikate sind also in der PL in gewisser Weise die Entsprechung zu den Aussagenvariablen der AL
- Sie können mit Junktoren verknüpft werden (mehr dazu später).

Quantoren

- Allquantor: $\bigwedge$ (Bedeutung: *für alle...*)
- Existenzquantor: $\bigvee$ (Bedeutung: *es existiert (mindestens) ein...*)

Dabei wird angegeben über welche Variablen quantifiziert wird:

- $\bigwedge_x$ (Bedeutung: für alle x...)
- $\bigvee_y$ (Bedeutung: es gibt ein y...)

Beispiel: Für alle x gilt $P(x)$:

$$\bigwedge_x P(x)$$

Beispiel

Interpretation prädikatenlogischer Formeln

$x \ldots$ Lebewesen
$F(x) \ldots$ ist ein Fisch
$S(x) \ldots$ kann schwimmen

Was bedeuten die folgenden prädikatenlogischen Formeln:

1. $\bigwedge_x (F(x) \to S(x))$ alle Fische können schwimmen
2. $\bigwedge_x F(x) \to S(x)$ Wenn alle Lebewesen Fische sind, dann kann x schwimmen
3. $\bigwedge_x F(x) \to \bigwedge_x S(x)$ Wenn alle Lebewesen Fische sind (wären), dann könn(t)en alle Lebewesen schwimmen

Notation der Quantoren

- Der Allquantor $\bigwedge_x$ sieht der logischen Konjunktion sehr ähnlich.
- *Beispiel:* $\bigwedge_x P(x)$ hat die Bedeutung: für alle x gilt das Prädikat $P(x)$
- Vergleich zu Aussagenlogik:

  Die Formel $p_1 \wedge p_2 \wedge \ldots \wedge p_n$ ist genau dann wahr wenn *für alle* p_i mit $i = 1, \ldots, n$ gilt, dass sie wahr sind.

  Die Formel $p_1 \vee p_2 \vee \ldots \vee p_n$ ist wahr wenn ein p_i mit Wert wahr *existiert* ($i = 1, \ldots, n$).

Beispiel

Negation und Quantoren

$x \ldots$ Menschen
$S(x) \ldots$ ist sterblich

Was bedeuten die folgenden prädikatenlogischen Formeln:

1. $\bigwedge_x S(x)$ ... alle Menschen sind sterblich
2. $\neg \bigwedge_x S(x)$ ... nicht alle Menschen sind sterblich
3. $\bigwedge_x \neg S(x)$ ... alle Menschen sind unsterblich
4. $\neg \bigwedge_x \neg S(x)$ ... nicht alle Menschen sind unsterblich

Beispiel

Mehrstellige Prädikate

$x, y \ldots$ Menschen
$K(x, y) \ldots x$ kennt y, bzw. y wird von x gekannt

Was bedeuten die folgenden prädikatenlogischen Formeln:

1. $\bigwedge_x \bigvee_y K(x, y)$ ... jeder kennt jemanden
2. $\bigvee_y \bigwedge_x K(x, y)$ ... es gibt jemanden der von allen gekannt wird

Prioritäten und Klammerung

- Quantoren haben höhere Priorität als Junktoren
- **Folgerung:** Wenn ein Quantor auf mehrere (logisch verknüpfte) Prädikate wirken soll, müssen Klammern gesetzt werden!
- *Beispiel:*

$$\bigwedge_x F(x) \to S(x)$$

Hier wirkt der Allquantor nur auf $F(x)$, aber nicht auf $S(x)$

- *Beispiel:*

$$\bigwedge_x (F(x) \to S(x))$$

Der Quantor wirkt auf $F(x)$ und $S(x)$

Freie und gebundene Variable

- Eine vorkommende Variable (in einem Prädikat) auf die ein Quantor wirkt heißt **gebundene Variable**.
- Variablen die nicht im Geltungsbereich eines Quantors liegen heißen **freie Variablen** (oder ungebundene Variablen).

Geltungsbereich, freie und gebundene Variablen

Auf welche Variablen wirken die Quantoren?
Welche Variablen sind frei?

$$\bigwedge_x \Big(\bigvee_y (P(x, y) \wedge Q(x, y, z)) \to R(y, x) \Big) \leftrightarrow P(x, z)$$

In $P(x, y)$ sowie $Q(x, y, z)$ sind x und y gebunden, in $Q(x, y, z)$ ist z frei.
In $R(y, x)$ ist y gebunden und x frei.
Die Variablen in $P(x, z)$ sind jeweils ungebunden (frei).

Regeln von De Morgan

Sei $x \in \{x_1, x_2, x_3, \ldots, x_n\}$. Es gelten nun folgende Umformungen:

$$\bigwedge_x P(x) \Leftrightarrow P(x_1) \wedge P(x_2) \wedge \ldots \wedge P(x_n)$$
$$\Leftrightarrow \neg(\neg P(x_1) \vee \neg P(x_2) \vee \ldots \vee \neg P(x_n)) \Leftrightarrow \neg \bigvee_x \neg P(x)$$

Regeln von De Morgan

$$\bigwedge_x P(x) \equiv \neg \bigvee_x \neg P(x)$$

$$\bigvee_x P(x) \equiv \neg \bigwedge_x \neg P(x)$$

- In der praktischen Anwendung werden gängige Mathematische Relationen ($<, \leq, =, >, \geq$) oft direkt verwendet, und nicht explizit als Prädikat geschrieben. Wir verwenden jedoch ausschließlich die explizite Schreibweise als Prädikat.
- Prädikate werden meist mit positiver Bedeutung gewählt. Z.B. $S(x)$ für kann schwimmen, und somit $\neg S(x)$ kann *nicht* schwimmen.
- Funktionen können *geschachtelt* werden, wenn dies der Definitions- und Wertebereich zulässt (z.B. $f(g(x))$)
- Prädikate können **nicht** geschachtelt werden!
- In der PL 2. Stufe kann auch über Prädikate quantifiziert werden.
- Interpretationen und Modelle befassen sich mit der Bestimmung von Subjektbereichen und Bedeutungen von Prädikaten/Formeln bei gegebenen prädikatenlogischen Formeln.

- In diesem Abschnitt sind einige Beispiele für Modellierungen mittels der PL zusammengestellt.
- Ziel ist eine möglichst präzise Umsetzung der Angabe.
- Folgende Dinge seien jedoch angemerkt:
  - Die Umsetzung ist nicht immer ganz einfach, und erfordert Übung!
  - Die Umsetzung ist *nicht eindeutig*:
    - Die Wahl des Subjektbereiches ermöglicht Einschränkungen vorab, die alternativ durch Prädikate formuliert werden können.
    - Die Kombination mehrerer Prädikate durch Junktoren kann oft durch eine geringere Anzahl an mehrstelligen Prädikaten ersetzt werden.

Modellieren Sie mittels PL:

"Keiner der blauen Elefanten kann programmieren, außer die fliegenden Elefanten"

$x \ldots$ Elefanten
$B(x) \ldots$ ist blau
$F(x) \ldots$ kann fliegen
$P(x) \ldots$ kann programmieren

$$\neg \bigvee_x \Big(B(x) \wedge P(x) \wedge \neg F(x) \Big)$$

Modellieren Sie mittels PL:

"Wenn alle Strichpunkte Gitterpunkte sind, dann gibt es keinen Randpunkt der auch Knotenpunkt ist."

$$\bigwedge_x \Big(S(x) \to G(x)\Big) \to \neg \bigvee_x \Big(R(x) \wedge K(x)\Big)$$

Modellieren Sie mittels PL:

"Nicht alle Anwälte gewinnen einen Prozess, obwohl sie weder schlecht vorbereitet sind, noch einen Verbrecher verteidigen."

$x \ldots$ Anwälte
$y \ldots$ Verbrecher
$P(x) \ldots$ x gewinnt Prozess
$S(x) \ldots$ x ist schlecht vorbereitet
$K(x, y) \ldots$ x verteidigt y

$$\neg \bigwedge_x \bigvee_y \Big(P(x) \wedge \neg S(x) \wedge \neg K(x, y)\Big)$$

Modellieren Sie mittels PL:

Formulieren Sie folgenden Satz mittels Prädikatenlogik:

Alle Personen die gleichzeitig eine Sonnenfinsternis beobachten benötigen eine eigene Finsternisbrille.

$m, n \ldots$ Menschen
$x, y \ldots$ Finsternisbrille
$B(m, n) \ldots$ m beobachtet gleichzeitig mit n Sonnenfinsternis
$F(m, x) \ldots$ m hat Finsternisbrille x
$G(a, b) \ldots$ a ist gleich b

$$\bigwedge_{m,n} \bigvee_{x,y} \Big(B(m, n) \wedge \neg G(m, n) \to (F(m, x) \wedge F(n, y) \wedge \neg G(x, y))\Big)$$

Modellieren Sie mittels PL:

Formulieren Sie folgenden Satz mittels Prädikatenlogik:

Es gibt Maschinen, die deshalb nicht laufen, weil sie entweder kaputt sind, oder sie niemand bedient.

$x \ldots$ Maschinen
$m \ldots$ Menschen, die Maschinen bedienen können
$L(x) \ldots$ x läuft
$K(x) \ldots$ x ist kaputt
$B(x, y) \ldots$ x verteidigt y

$$\bigvee_x \neg \bigvee_m \Big(\neg L(x) \wedge \neg(K(x) \leftrightarrow B(m, x))\Big)$$

Modellieren Sie mittels PL:

Formulieren Sie den *Satz des Pythagoras* mittels der PL:

$x \ldots$ rechtwinkelige Dreiecke
$a, b, c \ldots$ Seiten eines Dreiecks
$K(a, x) \ldots$ a ist Kathete von x
$H(a, x) \ldots$ a ist Hypothenuse von x
$G(a, b) \ldots$ a ist gleich b
$l(a) \ldots$ Länge der Seite a
$q(v) = v^2$
$s(m, n) = m + n$

$$\bigwedge_{x,a,b,c} \left(K(a,x) \wedge K(b,x) \wedge H(c,x) \wedge \neg G(a,b) \rightarrow G\Big(s\Big(q\big(l(a)\big), q\big(l(b)\big)\Big), q\big(l(c)\big)\Big) \right)$$